中英對照譯本

來源：Beeck Center for Social Impact + Innovation 與 Massive Data Institute（喬治城大學），“AI-Powered Rules as Code: Experiments with Public Benefits Policy”，2025 年 3 月 24 日發表，刊載於 Digital Government Hub。

本譯本翻的版本：完整報告的網頁版全文。網址：https://digitalgovernmenthub.org/publications/ai-powered-rules-as-code-experiments-with-public-benefits-policy/ 。該頁面未提供 PDF 下載；另有摘要頁（…-policy-summary/），內容是此報告的縮短版，摘要檔另有說明。

範圍說明：報告網頁版全文約 1.4 萬英文字詞、20 張圖、28 張表，超過 30 頁等量，因此依規範處理如下。已完整翻譯：摘要、前言、專案概覽、實驗一至實驗四（動機、方法、發展與挑戰、發現、使用情境考量）、結論。表格：數字型表格（表 1 至 15、16 至 19、28 等）與引導式提示範本（表 27）保留英文原文，只翻表名與說明文字；表 20 至 26（逐題回應範例）與附錄的評分準則、試算表清單未收錄，僅列標題或一句話說明，請見原文。圖：保留原圖，圖內文字為英文，圖名已翻譯。參考文獻與致謝名單保持原樣。

體例：每個英文段落（黑字）之後接繁體中文譯文（藍字、左側藍線）。原文的「may／could／suggests」等保守用語依原意譯為「可能／或許／顯示」，未改成斷言。

譯者按：原文表 8 說明寫 “ChatGPT API”，依上下文應指 GPT-4o API，已在譯文標註。

術語表

英文	中文
Rules as Code	規則即程式碼
LLM（Large Language Model）	大型語言模型
RAG（Retrieval-Augmented Generation）	檢索增強生成
SNAP	補充營養援助計畫（SNAP）
Medicaid	聯邦醫療補助（Medicaid）
pseudocode	偽代碼
rules template	規則範本
chunk / embedding	區塊 / 嵌入向量
human in the loop	人類參與審查（人在迴路中）
BBCE	廣泛類別資格

AI-Powered Rules as Code: Experiments with Public Benefits Policy

AI 驅動的規則即程式碼（Rules as Code）：公共福利政策實驗

This report documents four experiments exploring if AI can be used to expedite the translation of SNAP and Medicaid policies into software code for implementation in public benefits eligibility and enrollment systems under a Rules as Code approach.

> 本報告記錄四項實驗，探討在「規則即程式碼」的做法下，AI 能否加快把 SNAP 與 Medicaid 政策轉譯成軟體程式碼，供公共福利資格審查與申請系統使用。

Abstract

摘要

Public interest technologists are still early in our understanding of how to effectively use large language models (LLMs) to translate policy into code. This report documents four experiments conducted to evaluate the current performance of commercially-available LLMs in translating policies into plain language summaries, machine-readable pseudocode, or usable code within a Rules as Code process. We used eligibility rules and policies for the Supplemental Nutrition Assistance Program (SNAP) and Medicaid. The experiments include asking a chatbot or LLM about specific policies, summarizing policy in a machine-readable format, and using fine-tuning or Retrieval-Augmented Generation (RAG) to enhance an LLM's ability to generate code that encodes policy. We found that LLMs are capable of supporting the process of generating code from policy, but still require external knowledge and human oversight within an iterative process for any policies containing complex logic.

> 公共利益科技工作者對於如何有效運用大型語言模型（LLM）把政策轉成程式碼，仍處於早期的理解階段。本報告記錄四項實驗，用來評估市售 LLM 目前在「規則即程式碼」流程中的表現，包括把政策轉成白話摘要、機器可讀的偽代碼（pseudocode），或可用的程式碼。我們使用的是補充營養援助計畫（SNAP）與聯邦醫療補助（Medicaid）的資格規則與政策。這些實驗包括：向聊天機器人或 LLM 詢問特定政策、把政策摘要成機器可讀格式，以及運用微調（fine-tuning）或檢索增強生成（RAG）來提升 LLM 生成政策編碼程式碼的能力。我們發現，LLM 能夠協助從政策產生程式碼的過程，但只要政策含有複雜邏輯，仍需要外部知識與人工監督，並在反覆迭代的流程中進行。

Introduction

前言

The rise of commercially-available large language models (LLMs) presents an opportunity to use artificial intelligence (AI) to expedite the translation of policies into software code for implementation in public benefits eligibility and enrollment systems under a Rules as Code approach. Our team experimented with multiple LLMs and methodologies to determine how well the LLMs could translate Supplemental Nutrition Assistance Program (SNAP) and Medicaid policies across seven different states. We present our findings in this report.

> 市售大型語言模型（LLM）的興起，帶來一個機會：可以運用人工智慧（AI），加快把政策轉譯成軟體程式碼，供公共福利資格審查與申請系統在「規則即程式碼」的做法下使用。我們的團隊以多種 LLM 與方法進行實驗，檢視這些 LLM 轉譯七個州的補充營養援助計畫（SNAP）與 Medicaid 政策的能力有多好。本報告呈現我們的發現。

We conducted initial experiments from June to September 2024 during the Policy2Code Prototyping Challenge. The challenge was hosted by the Digital Benefits Network and the Massive Data Institute, as part of the Rule as Code Community of Practice. Twelve teams from the U.S. and Canada participated in the resulting Policy2Code Demo Day. We finished running the experiments and completing the analysis from October 2024 to February 2025.

> 我們在 2024 年 6 月至 9 月的 Policy2Code 原型挑戰賽（Policy2Code Prototyping Challenge）期間進行了初步實驗。該挑戰賽由數位福利網絡（Digital Benefits Network）與大數據研究院（Massive Data Institute）主辦，屬於「規則即程式碼實踐社群」的一部分。來自美國與加拿大的十二個團隊參加了後續的 Policy2Code 成果展示日。我們在 2024 年 10 月至 2025 年 2 月完成實驗執行與分析。

In this report you will learn more about key takeaways including:

> 在本報告中，你將進一步了解以下重點：

- LLMs can help support the Rules as Code pipeline. LLMs can extract programmable rules from policy by leveraging expert knowledge retrieved from policy documents and employing well-crafted templates.

> • LLM 可以協助「規則即程式碼」的流程。LLM 能借助從政策文件中檢索到的專家知識，並搭配精心設計的範本，從政策中擷取出可程式化的規則。

- LLMs achieve better policy-to-code conversion when prompts are detailed and the policy logic is simple.

> • 當提示（prompt）寫得詳細、且政策邏輯單純時，LLM 在政策轉程式碼上的表現較好。

- State governments can make it easier for LLMs to use their policies by making them digitally accessible.

> • 州政府若把政策做成數位化、易於取用的形式，LLM 就更容易使用這些政策。

• Humans must be in the loop to review outputs from LLMs. Accuracy and equity considerations must outweigh efficiency in high-stakes benefits systems.

> ・必須有人類參與審查 LLM 的輸出。在高風險的福利系統中，正確性與公平性的考量必須優先於效率。

• Current web-based chatbots have mixed results, often risking incorrect information presented in a confident tone.

> ・目前以網頁為主的聊天機器人表現不一，常常以很有把握的口氣呈現錯誤資訊。

Prefer to read a summary of the AI-Powered Rules as Code: Experiments with Public Benefits Policy report?

> 想先看「AI 驅動的規則即程式碼：公共福利政策實驗」報告的摘要版嗎？

Overview of Experiments

實驗概覽

Our experiments addressed the following questions:

> 我們的實驗要回答下列問題：

• Experiment 1: How well can LLM chatbots answer general SNAP and Medicaid eligibility questions based on their training data and/or resources available on the internet? What factors affect their responses?

> ・實驗一：LLM 聊天機器人依據其訓練資料及／或網路上的資源，回答一般性的 SNAP 與 Medicaid 資格問題，能做到多好？哪些因素會影響它們的回答？

• Experiment 2: How well can an LLM generate accurate, complete, and logical summaries of benefits policy rules when provided official policy documents?

> ・實驗二：當提供官方政策文件時，LLM 能多好地產出準確、完整且合乎邏輯的福利政策規則摘要？

• Experiment 3: How well can an LLM extract machine-readable rules from unstructured policy documents in terms of output relevance and accuracy?

> ・實驗三：LLM 從非結構化的政策文件中擷取機器可讀規則，在輸出的相關性與準確度上能做到多好？

• Experiment 4: How effectively can an LLM generate software code to determine eligibility for public benefits programs?

> ・實驗四：LLM 生成用於判定公共福利資格的軟體程式碼，成效如何？

Below, we describe each of the four experiments, focusing on their motivation, methodology, findings, and considerations for public benefits use cases.

> 以下我們分別說明四項實驗，重點放在動機、方法、發現，以及對公共福利使用情境的考量。

Open Materials

公開資料

Each experiment includes openly accessible materials in a linked appendix. We share the rubrics we developed to evaluate the accuracy, completeness, and relevance of the responses. Additionally, we share our prompts, responses, and scores. Note that the prompts may generate different responses if run again through the same models because models are continually being updated.

> 每項實驗都在附錄中附上可公開取得的資料。我們分享了為評估回答的準確性、完整性與相關性所設計的評分準則，也分享了我們的提示、回應與分數。請注意，由於模型持續更新，同樣的提示再次送進同一個模型，可能會得到不同的回應。

Current State: Policy from Federal and State Government

現況：聯邦與州政府的政策

Public benefits in the United States are governed by a complex network of federal, state, territorial, tribal, and local entities. Governments enact a web of intersecting laws, regulations, and policies related to these programs. These are often communicated1 in lengthy and complex PDFs and documents distributed across disparate websites. Maintaining eligibility rules for public benefits across eligibility screeners, enrollment systems, and policy analysis tools—like calculating criteria for household size, income, and expenses—is already a significant cross-sector challenge. This complexity hinders government administrators' ability to deliver benefits effectively and makes it difficult for residents to understand their eligibility.

> 美國的公共福利由聯邦、州、領地、部落與地方等各層級單位組成的複雜網絡共同管轄。各級政府制定了彼此交錯的法律、法規與政策，這些內容通常以冗長又複雜的 PDF 及文件形式傳達[1]，散落在不同網站上。要在資格篩檢工具、申請系統與政策分析工具之間維持公共福利的資格規則（例如計算家戶人數、收入與支出的標準），本身就已是跨部門的重大挑戰。這種複雜性妨礙了政府行政人員有效提供福利，也讓居民難以了解自己是否符合資格。

We have heard directly from state and local government practitioners about the challenges2 with their eligibility and enrollment systems and how they intersect with policy. For example, one state practitioner described it as "a scramble" to update the code in legacy systems. Implementation is often completed by a policy team, technology team, or external vendor, which increases the potential for errors or inconsistencies when system changes or updates are needed. An additional challenge is that no U.S. government agency currently makes its eligibility code open source, which ultimately reduces transparency and adaptation across other government systems.

> 我們直接聽取了州與地方政府實務人員談到他們在資格審查與申請系統上的挑戰[2]，以及這些系統與政策如何交會。例如，有一位州的實務人員形容，更新舊系統中的程式碼就像「手忙腳亂」。系統的實作通常由政策團隊、技術團隊或外部廠商完成，這使得每當系統需要變更或更新時，出現錯誤或不一致的可能性就更高。另一項挑戰是，目前美國沒有任何政府機關將其資格審查程式碼開源，最終降低了各政府系統之間的透明度與可調適性。

Introduction to Rules as Code

規則即程式碼簡介

Adopting Rules as Code for public benefits is a crucial strategy to improve the connection between policy and digital service delivery in the U.S. The Organization for Economic Co-operation and Development (OECD) defines3 Rules as Code as “an official version of rules (e.g. laws and regulations) in a machine-consumable form, which allows rules to be understood and actioned by computer systems in a consistent way.” Using Rules as Code in expert systems and automated or semi-automated decision-making systems is a key use case4.

> 在公共福利領域採用「規則即程式碼」，是改善政策與數位服務提供之間連結的關鍵策略。經濟合作暨發展組織（OECD）將「規則即程式碼」定義為[3]：「規則（例如法律與法規）的官方版本，以機器可讀取的形式呈現，使電腦系統能以一致的方式理解並執行這些規則。」把「規則即程式碼」用在專家系統，以及自動或半自動的決策系統，是一項關鍵的應用情境[4]。

This approach facilitates seamless and transparent policy integration into standardized public benefits delivery systems. A state leader noted that Rules as Code supports a “no wrong door” vision, ensuring consistent eligibility criteria across all service entry points, or “doors.”

> 這種做法有助於把政策順暢且透明地整合進標準化的公共福利提供系統。一位州的領導者指出，「規則即程式碼」支持「沒有錯誤的門」（no wrong door）願景，也就是確保在所有服務入口（也就是各個「門」），資格標準都一致。

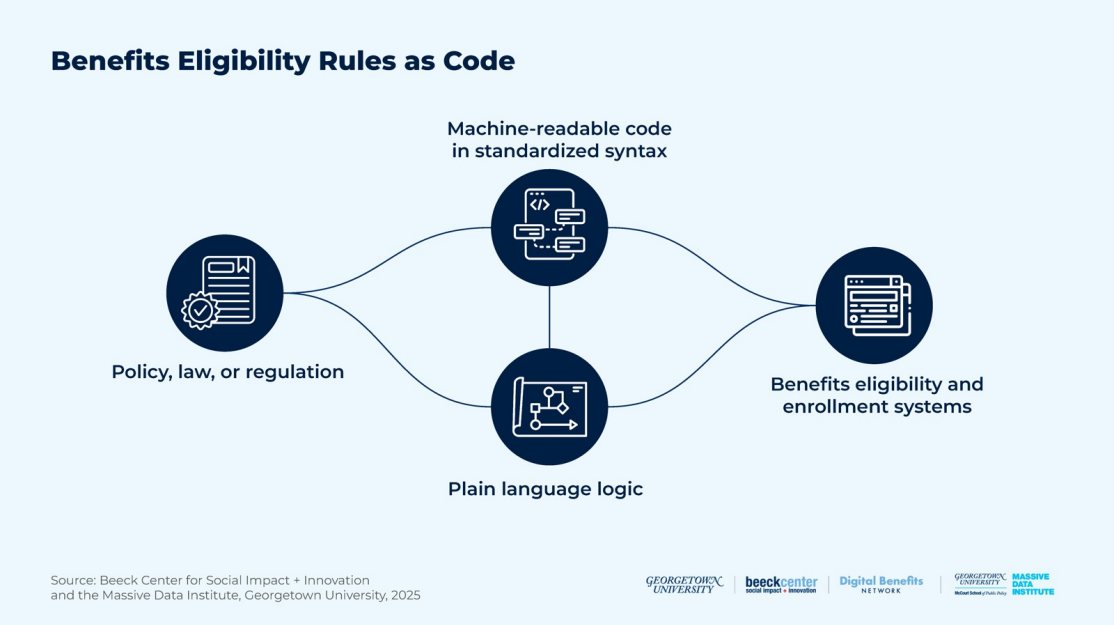
In a future Rules as Code approach to benefits eligibility, policies and regulations would be translated into both plain-language logic flows (pseudocode) and standardized, machine-readable formats, such as Extensible Markup Language (XML), JavaScript Object Notation (JSON), or Yet Another Markup Language (YAML). This would enable easy access by various systems, including benefits applications, eligibility screeners, and policy analysis tools. Doing this would enable stakeholders to simultaneously review the legislation or regulation, plain language logic, and code, and know that all are official and ready for implementation. Ideally, this would also enable a more seamless cycle of writing, testing, and revising the rules. Others have written about the challenges and consequences of implementing Rules as Code, including an in-depth analysis5 of work in New Zealand, and a lawyer and programmer’s recreation of government system code6 for French housing benefits.

> 在未來以「規則即程式碼」處理福利資格的做法中，政策與法規會被轉譯成兩種形式：白話的邏輯流程（偽代碼），以及標準化的機器可讀格式，例如可延伸標記語言（XML）、JavaScript 物件表示法

（JSON）或 YAML。這將讓各種系統，包括福利申請系統、資格篩檢工具與政策分析工具，都能輕易取用。這樣做可以讓利害關係人同時檢視法條或法規、白話邏輯與程式碼，並確知三者都是官方版本、可供實施。理想上，這也會讓撰寫、測試與修訂規則的循環更為順暢。其他人已寫過實施「規則即程式碼」的挑戰與後果，包括對紐西蘭相關工作的深入分析[5]，以及一位律師兼程式設計師重建法國住房補助政府系統程式碼的案例[6]。

Figure 1: Benefits Eligibility Rules as Code

圖 1：福利資格的規則即程式碼



There is a vibrant ecosystem of organizations that have already created online eligibility screening tools, benefits applications, policy analysis tools, and open-source resources across numerous public benefits programs. These organizations play a crucial role in increasing access to benefits for eligible individuals, and several serve as tangible examples of how open, standardized code can translate policy into system rules.

目前已有蓬勃的組織生態系，針對眾多公共福利計畫建立了線上資格篩檢工具、福利申請系統、政策分析工具與開源資源。這些組織在提高符合資格者取得福利的機會上扮演關鍵角色，其中數個組織也成為具體範例，顯示開放、標準化的程式碼如何把政策轉譯成系統規則。

In addition to open-code bases, there are other frameworks that can inform the standardization of communicating Rules as Code for U.S. public benefits programs. To reduce time and money spent on creating new frameworks, we believe it is important to evaluate and test any existing frameworks or standards to determine if they can be adopted or further developed.

> 除了開源的程式碼庫之外，還有其他框架可以作為美國公共福利計畫在傳達「規則即程式碼」時標準化的參考。為了減少建立新框架所花費的時間與金錢，我們認為重要的是評估並測試現有的框架或標準，判斷它們是否能被採用或進一步發展。

Introduction to Generative AI

生成式 AI 簡介

Artificial intelligence (AI) is the science of developing intelligent machines. Machine learning is a branch of AI that learns to make predictions based on training data. Deep learning is a specific type of machine learning, inspired by the human brain, that uses artificial neural networks to make predictions. The goal is for each node in the network to process information in a way that imitates a neuron in the brain. The more layers of artificial neurons in the network, the more patterns the neural network is able to learn and use for prediction.

> 人工智慧（AI）是研發智慧機器的科學。機器學習是 AI 的一個分支，從訓練資料中學習並進行預測。深度學習是機器學習中的一種特定類型，受人腦啟發，使用人工神經網路進行預測。其目標是讓網路中的每個節點以模仿腦中神經元的方式處理資訊。網路中的人工神經元層數越多，神經網路能學習並用於預測的模式就越多。

Many successful generative AI tools and applications are designed using deep learning models. A large difference between classic machine learning and generative AI is that generative AI models not only identify patterns that they have learned, but also generate new content. One specific type of deep learning model for processing human text is Large Language Models (LLMs). The goal of LLMs is to attempt to capture the relationship between words and phrases. LLMs have been trained using enormous amounts of text and have human-like language ability across a range of traditional language tasks. However, it remains unclear how well different LLMs and tools built using LLMs—e.g. chatbots—work in more complex, domain-specific prediction tasks, like Rules as Code.

> 許多成功的生成式 AI 工具與應用，都是以深度學習模型設計而成。傳統機器學習與生成式 AI 的一大差異在於，生成式 AI 模型不只是辨識它學到的模式，還會生成新的內容。處理人類文字的深度學習模型中，有一種特定類型是大型語言模型（LLM）。LLM 的目標是嘗試捕捉字詞與片語之間的關係。LLM 以海量文字訓練而成，在一系列傳統語言任務上具備接近人類的語言能力。然而，不同的 LLM，以及用 LLM 打造的工具（例如聊天機器人），在更複雜、屬於特定領域的預測任務（例如「規則即程式碼」）上表現如何，目前仍不清楚。

Intersection of Generative AI with Rules as Code

生成式 AI 與規則即程式碼的交會

As the use of automated and generative AI tools increases and more software attempts to translate benefits policies into code, the existing pain points will likely be exacerbated. To increase access for eligible individuals and ensure system accuracy, it is imperative to quickly build a better understanding of how generative AI technologies can, or cannot, aid in the translation and standardization of eligibility rules in software code for these vital public benefits programs. We see a critical opportunity to create open, foundational resources for benefits eligibility logic that can inform new AI technologies and ensure that they are correctly interpreting benefits policies.

> 隨著自動化與生成式 AI 工具的使用增加，以及愈來愈多軟體嘗試把福利政策轉譯成程式碼，現有的痛點很可能被加劇。為了讓符合資格者更容易取得福利、並確保系統的準確性，當務之急是盡快更深入了解：生成式 AI 技術在把資格規則轉譯並標準化為軟體程式碼上，能夠或無法提供什麼協助。我們認為這是一個關鍵契機，可以為福利資格邏輯建立開放的基礎資源，用以指引新的 AI 技術，確保它們正確詮釋福利政策。

If generative AI tools are able to effectively translate public policy or program rules into code and/or convert code between software languages7, it could:

> 如果生成式 AI 工具能有效地把公共政策或計畫規則轉譯成程式碼，及／或在不同程式語言之間轉換程式碼[7]，它可能：

- Speed up a necessary digital transformation to Rules as Code;

> - 加速推動勢在必行的「規則即程式碼」數位轉型；

- Allow for humans to check for accuracy and act as logic and code reviewers instead of programmers;

> - 讓人類改當邏輯與程式碼的審查者來檢查準確性，而不必擔任程式設計者；

- Automate testing to validate code;

> - 自動化測試以驗證程式碼；

- Allow for easier implementation of updates and changes to rules;

> - 讓規則的更新與變更更容易實施；

- Create more transparency in how rules operate in digital systems;

> - 提高規則在數位系統中運作方式的透明度；

- Increase efficiency by eliminating duplicate efforts across multiple levels of government and delivery organizations; and

> - 消除多個政府層級與服務提供單位之間的重複工作，提升效率；以及

- Reduce the burden on public employees administering and delivering benefits.

> - 減輕公務人員在管理與提供福利時的負擔。

Additionally, once rules are in code, it could enable new pathways for policymakers, legislators, administrators, and the public to model and measure the impacts of policy changes. For example, a generative AI tool could help model different scenarios for a rules change, using de-identified or synthetic data to measure impact on a specific population or geography. Additionally, generative AI can also help governments migrate code from legacy systems and translate it into modern code languages or syntaxes, as well as enable interoperability between systems.

> 此外，一旦規則變成程式碼，政策制定者、立法者、行政人員與民眾就可能有新的途徑，來模擬並衡量政策變動的影響。舉例來說，生成式 AI 工具可以協助模擬規則變更的不同情境，利用去識別化或合成資料，衡量對特定人口或地區的影響。此外，生成式 AI 也可以協助政府從舊系統遷移程式碼，並轉譯成現代的程式語言或語法，同時促成系統之間的互通性。

How We've Been Advancing Rules as Code

我們如何推進規則即程式碼

The Digital Benefits Network (DBN) and Massive Data Institute (MDI) teams are among the first to extensively research how to apply a Rules as Code framework to the U.S. public benefits system. We draw on international research and examples, and have identified numerous U.S.-based projects that could inform a national strategy, along with a shared syntax and data standard.

> 數位福利網絡（DBN）與大數據研究院（MDI）團隊，是最早廣泛研究如何把「規則即程式碼」框架應用於美國公共福利系統的團隊之一。我們參考國際研究與案例，並找出許多美國境內的專案，可作為國家級策略以及共用語法與資料標準的參考。

The DBN hosts the Rules as Code Community of Practice—a shared space for people working on public benefits eligibility and enrollment systems, particularly people tackling how policy becomes software code.

> DBN 主辦「規則即程式碼實踐社群」，這是一個供從事公共福利資格審查與申請系統的人員共享的空間，特別是那些處理「政策如何變成軟體程式碼」的人。

Project Overview

專案概覽

Policy Focus

政策焦點

We focused on SNAP and Medicaid because they are essential benefits programs with high participation rates and are often integrated into a combined application. Both are means-tested programs that use specific criteria to determine eligibility and benefits amounts.

> 我們聚焦於 SNAP 與 Medicaid，因為它們是參與率高的重要福利計畫，且經常整合在同一份申請書中。兩者都是有資產／所得審查（means-tested）的計畫，以特定標準判定資格與福利金額。

Broadly, benefits policy rules fall into two categories: eligibility rules and benefits rules. Eligibility rules determine whether an applicant qualifies for the program based on their personal and household information. On the other hand, benefits rules refer to the type and scope of benefits an eligible applicant is entitled to receive. Both categories of rules are essential for developing scalable applications that improve and streamline the digitized delivery of benefits.

> 大致上，福利政策規則可分為兩類：資格規則與福利規則。資格規則依申請人的個人與家戶資訊，判定申請人是否符合計畫資格。另一方面，福利規則指的是符合資格的申請人有權獲得的福利類型與範圍。這兩類規則對於開發可擴展的應用程式、改善並簡化福利的數位化提供，都不可或缺。

States

州別

We focused on SNAP and Medicaid policies in five states: California, Georgia, Michigan, Pennsylvania, and Texas. Additionally, we included Alaska's SNAP policy and Oklahoma's Medicaid policy.

> 我們聚焦於五個州的 SNAP 與 Medicaid 政策：加州、喬治亞州、密西根州、賓夕法尼亞州與德州。此外，我們還納入阿拉斯加州的 SNAP 政策與奧克拉荷馬州的 Medicaid 政策。

We selected these states based on the following criteria, aiming for diversity across these characteristics:

> 我們依下列標準選擇這些州，目標是在這些特性上具備多樣性：

- The state's previous identification as delivering "strong" or "poor" communication of eligibility criteria in SNAP policy manuals

> - 該州先前被認定在 SNAP 政策手冊中，對資格標準的溝通屬於「良好」或「不佳」

- Political party affiliation of the state's governor

> - 該州州長的政黨屬性

- The state's ability to offer Broad-Based Categorical Eligibility (BBCE) for SNAP

> - 該州是否能提供 SNAP 的廣泛類別資格（Broad-Based Categorical Eligibility，BBCE）

- The state's participation in the Affordable Care Act (ACA) expansion for Medicaid

> - 該州是否參與《平價醫療法案》（ACA）的 Medicaid 擴大

Table 1: State Selection Criteria

表 1：州別選擇標準（表格內容保留英文原文）

State	Strong/Poor SNAP Manual	Governor's Political Party (as of October 2024)	BBCE Offered for SNAP?	ACA Expansion?
Alaska	Strong	Republican	No	N/A (only looked at SNAP policy)
California	Unknown	Democrat	Yes	Yes
Georgia	Poor	Republican	Yes	No
Michigan	Unknown	Democrat	Yes	Yes
Oklahoma	Unknown	Republican	N/A (Only looked at Medicaid policy)	Yes (recent expansion)
Pennsylvania	Strong	Democrat	Yes	Yes
Texas	Unknown	Republican	Yes, with asset limit on Temporary Assistance for Needy Families (TANF)/Maintenance of Effort (MOE) programs	No

Technology Focus

技術焦點

Our four experiments used a well-known LLM, GPT-4o, accessed through the Application Programming Interface (API). We also used the web browser versions of two AI chatbots: ChatGPT and Gemini. At the time of this study, ChatGPT used GPT 3.5 while Gemini used Gemini 1.5 Flash. In this study, we refer to the LLM and chatbot output as LLM output. We recognize that chatbots have less flexibility with respect to setting parameters.

> 我們的四項實驗使用一個知名的 LLM：GPT-4o，並透過應用程式介面（API）存取。我們也使用兩種 AI 聊天機器人的網頁瀏覽器版本：ChatGPT 與 Gemini。在本研究進行時，ChatGPT 使用 GPT 3.5，Gemini 使用 Gemini 1.5 Flash。本研究中，我們把 LLM 與聊天機器人的輸出統稱為 LLM 輸出。我們知道，聊天機器人在設定參數上的彈性較小。

Assessing LLM Outputs

評估 LLM 的輸出

Our research group developed rubrics to evaluate the responses the LLMs generated in Experiments 1 and 2. Each category—accuracy, completeness, relevance, and state-specific or current applicability—captured a different aspect of the response. Moreover, each factor influenced the degree to which the response met the requirements of the prompt. For example, a response might have been highly accurate, but lacked relevance, making it less effective for specific prompts. By evaluating these categories separately, we were able to better assess the quality of LLM-generated responses.

> 我們的研究小組為實驗一與實驗二設計了評分準則，用來評估 LLM 產生的回應。每個類別，也就是準確性、完整性、相關性，以及州別特定性或現行適用性，各自捕捉回應的不同面向。此外，每個因素都會影響回應符合提示要求的程度。例如，一則回應可能非常準確，但缺乏相關性，使它對特定提示的成效較差。藉由分別評估這些類別，我們能更好地評估 LLM 所生成回應的品質。

Additionally, human evaluation—often considered the gold standard—is typically slow and costly. For Experiments 1 and 2, our human evaluators were a small group of two people who, while knowledgeable when reviewing the documents, were not experts in the state policies. Moving forward, it would be valuable to have established guidelines or evaluation metrics to support this type of task. Also, having a larger and more diverse group of evaluators could improve evaluation consistency and reduce subjectivity in scoring.

> 此外，人工評估雖常被視為黃金標準，但通常既慢又昂貴。在實驗一與實驗二中，我們的人工評估者是兩人小組，他們在審閱文件時雖具備相當知識，卻不是各州政策的專家。往後若能有既定的指引或評估指標來支援這類任務，將會很有價值。此外，擁有更多、更多元的評估者，也可以提升評估的一致性並降低評分的主觀性。

For Experiment 3, we evaluated the LLM's structured rule outputs (e.g., income limits or work requirements) by comparing them against manually-verified values from state policy documents. This helped us quantify the alignment between generated rules and primary sources. In Experiment 4, we adopted a qualitative approach, evaluating the quality and utility of AI-generated code through criteria such as output accuracy (correct implementation of policy logic), logical consistency (coherence of decision flows), and rules coverage (completeness of encoded policy requirements).

> 在實驗三中，我們把 LLM 輸出的結構化規則（例如收入上限或工作要求），與人工核實過的州政策文件數值進行比對。這有助於量化所生成規則與原始來源的吻合程度。在實驗四中，我們採取質性方法，依下列標準評估 AI 所生成程式碼的品質與效用：輸出正確性（政策邏輯是否正確實作）、邏輯一致性（決策流程是否連貫），以及規則涵蓋率（所編碼的政策要求是否完整）。

Experiment 1: Asking Chatbots About Benefits Eligibility

實驗一：向聊天機器人詢問福利資格

Motivation or Question

動機或問題

In this research, we set out to analyze how three LLMs—the web browser versions of ChatGPT and Gemini, as well as the GPT-4o API—extracted and provided information about eligibility for different policies. Our goal was to measure how well these models answered questions related to benefits programs like SNAP and Medicaid. The main questions guiding our study were:

> 在這項研究中，我們著手分析三個 LLM（ChatGPT 與 Gemini 的網頁瀏覽器版本，以及 GPT-4o API）如何擷取並提供不同政策的資格資訊。我們的目標是衡量這些模型回答 SNAP 與 Medicaid 等福利計畫相關問題的能力。引導本研究的主要問題為：

- How well can AI chatbots answer general SNAP and Medicaid eligibility questions?

> ・ AI 聊天機器人回答一般性的 SNAP 與 Medicaid 資格問題，能做到多好？

- What factors affect their responses?

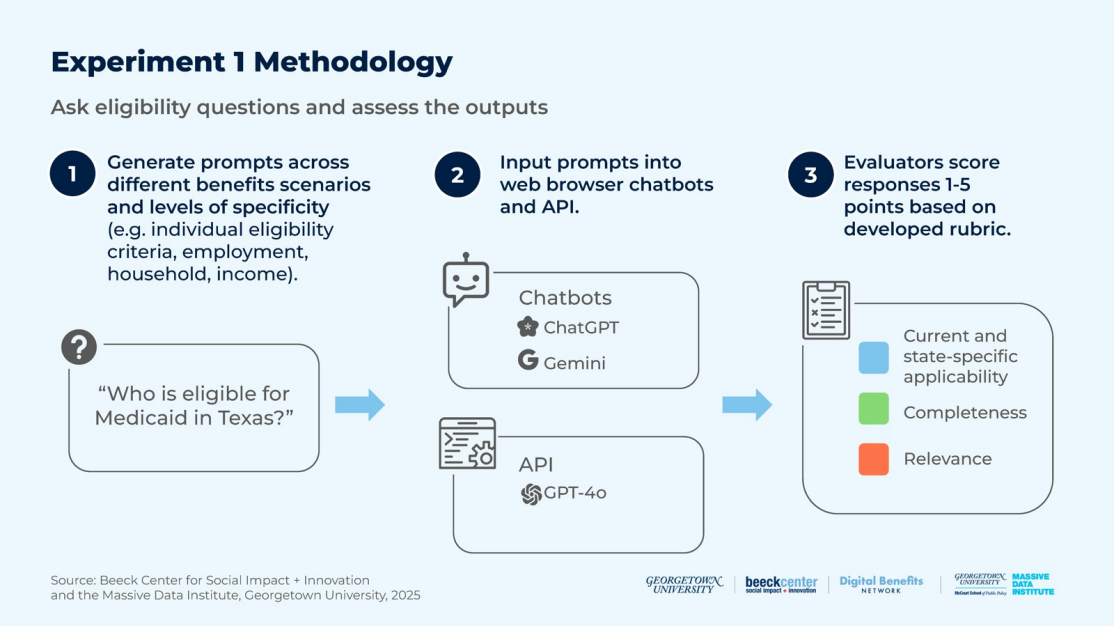
> ・ 哪些因素會影響它們的回應？

Methodology

方法

Figure 2: Experiment 1 Methodology

圖 2：實驗一方法



1. Prompt Generation

1. 產生提示

We took two different approaches to designing the prompts in the experiment. First, we used zero-shot prompting, which gave the model a prompt on a topic it had never seen or been trained on. We also used general knowledge prompting, an advanced technique where questions were structured to leverage the model's broad understanding of the world. Read more about different prompting techniques here8.

> 我們在實驗中以兩種不同方式設計提示。首先，我們使用零樣本提示（zero-shot prompting），也就是對模型提出一個它從未見過或未曾受過訓練的主題的提示。我們也使用一般知識提示（general knowledge prompting），這是一種進階技巧，把問題的結構設計成能運用模型對世界的廣泛理解。關於不同提示技巧的更多說明，請見此處[8]。

We divided the prompts into the following categories:

> 我們把提示分為下列類別：

- "Who is eligible for Medicaid in Texas?"
  - 「在德州，誰有資格領取 Medicaid？」
- "What is the income criteria for Medicaid eligibility in Texas?"
  - 「在德州，Medicaid 資格的收入標準是什麼？」
- "What is the employment criteria for Medicaid eligibility in Texas?"
  - 「在德州，Medicaid 資格的就業標準是什麼？」
- "I am a single adult that cannot work due to a medical condition, am I eligible for SNAP in Georgia?"
  - 「我是單身成年人，因健康狀況無法工作，我在喬治亞州有資格領取 SNAP 嗎？」
- "I do not work, am I eligible for SNAP in Michigan?"
  - 「我沒有工作，我在密西根州有資格領取 SNAP 嗎？」
- "I am a foster parent with foster children, are we eligible for SNAP in Pennsylvania?"
  - 「我是有寄養兒童的寄養家長，我們在賓夕法尼亞州有資格領取 SNAP 嗎？」
- "Am I eligible for SNAP in Alaska if my gross monthly income is $1,975 and my net monthly income is $1,760?"
  - 「如果我的每月總收入是 1,975 美元、每月淨收入是 1,760 美元，我在阿拉斯加州有資格領取 SNAP 嗎？」
- "Is someone eligible for SNAP if they are enrolled in TANF in Alaska?"
  - 「在阿拉斯加州，已加入 TANF 的人有資格領取 SNAP 嗎？」
- "I am pregnant woman, what eligibility requirements do I have to meet to qualify for SNAP in Alaska?"
  - 「我是孕婦，在阿拉斯加州要符合 SNAP 資格需要滿足哪些資格要求？」
- "I receive a non-cash MOE benefit. Do I qualify for SNAP in California?"
  - 「我領取非現金的 MOE 福利。我在加州有資格領取 SNAP 嗎？」

2. Assessing LLM Outputs

2. 評估 LLM 的輸出

For this experiment, our research group analyzed and scored the responses generated by the LLMs for state-specific and current applicability, relevance, and completeness.

> 在這項實驗中，我們的研究小組針對 LLM 生成的回應，分析並評分其州別特定性與現行適用性、相關性與完整性。

Table 2: Experiment 1 Rubric

表 2：實驗一評分準則（表格內容保留英文原文）

Current and state-specific applicability	**Is the information from the response current and state-specific?**	**Score 1-5**
Completeness	Is the response thorough and does it cover all elements requested in the prompt?	Score 1-5
Relevance	Is the response focused on the question, without adding irrelevant or unnecessary details?	Score 1-5

Read more about the rubric for Experiment 1 in the Appendix. →

> 關於實驗一評分準則的更多內容，請見附錄。→（譯按：準則三項各 1 至 5 分。「現行與州別特定適用性」：資訊是否為現行且屬於該州；「完整性」：回應是否周全、涵蓋提示要求的所有要素；「相關性」：回應是否聚焦於問題。）

Developments and Challenges

發展與挑戰

Three features of the LLMs affected the data used to generate their responses and our ability to evaluate their data sources.

> LLM 有三項特性會影響生成回應所用的資料，以及我們評估其資料來源的能力。

- Internet access: The web browser versions of ChatGPT and Gemini were able to search and present data from the internet in responses, whereas the GPT-4o API relied solely on its training data.

> • 網路存取：ChatGPT 與 Gemini 的網頁瀏覽器版本能搜尋網路資料並呈現在回應中，而 GPT-4o API 僅依賴其訓練資料。

• “Short-term” memory: The web browser version of ChatGPT included a setting to enable or disable short-term memory. When enabled, the model used information from previous prompts to inform responses. Our experiment aimed to evaluate how well the models answered SNAP and Medicaid policy questions using single prompts. However, a small number of ChatGPT responses showed that the short-term memory setting was on due to our error. Those responses remain in our analysis.

> • 「短期」記憶：ChatGPT 網頁瀏覽器版本有一項可開啟或關閉短期記憶的設定。開啟時，模型會用先前提示的資訊來影響回應。我們的實驗目的是評估模型在單一提示下回答 SNAP 與 Medicaid 政策問題的表現。然而，有少數 ChatGPT 的回應顯示，由於我們的疏失，短期記憶設定是開啟的。這些回應仍保留在我們的分析中。

• Citing sources: The web-browser versions of ChatGPT and Gemini were able to cite sources for their responses, whereas the GPT-4o API was not. Most of the responses in our experiment did not include citations, which made it challenging to determine the data sources used in responses.

> • 引用來源：ChatGPT 與 Gemini 的網頁瀏覽器版本能為其回應引用來源，而 GPT-4o API 不能。我們實驗中的大多數回應並未附上引用，這使得判斷回應所用的資料來源相當困難。

Findings

發現

Our findings for response performance across the three rubric categories (current/state-specific applicability, completeness, and relevance of response) are included in Experiment 1 Materials. It includes both “average performance scores” based on the rubric and “average percentage performance scores,” which convert average rubric scores into percentages for standardized comparisons of results across settings.

> 三個評分類別（現行／州別特定適用性、完整性、回應相關性）上的回應表現發現，收錄於「實驗一資料」。其中同時包含依評分準則計算的「平均表現分數」，以及把平均分數轉成百分比、以便在不同設定間標準化比較的「平均表現百分比分數」。

The spreadsheet is organized into tables that break down the results in the following ways:

> 該試算表整理成下列幾種拆解結果的表格：

Average Scores by Prompt Topic

依提示主題的平均分數

Table 3: Average Scores by Prompt Topic

表 3：依提示主題的平均分數（表格內容保留英文原文）

	State-		Completenes		Relevance of	

	Specific/ Current Applicability		**s**		**Response**	
Prompt Topic	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
Overall Eligibility Criteria	4.2	84%	2.0	40%	4.5	90%
Individual Eligibility Criteria	4.1	82%	3.1	62%	4.0	80%
Employment Scenarios	4.3	86%	2.6	52%	4.9	98%
Household Scenarios	4.5	90%	2.7	54%	5.0	100%
Income Scenarios	3.8	76%	2.6	52%	4.8	96%
Other Program Scenarios	4.5	90%	3.1	62%	4.9	98%
Other Scenarios	4.1	82%	2.9	58%	4.8	96%

This table breaks the scores down by prompt topics.

> 此表依提示主題拆解分數。

Notable results in this table include:

> 此表中值得注意的結果包括：

• Individual Eligibility Criteria prompts performed better on completeness than Overall Eligibility Criteria prompts. This suggests that the LLMs were less effective when asked to identify all of the eligibility criteria for SNAP or Medicaid in a single query, compared to when asked to identify the details of each individual criterion for eligibility in a single query.

> • 在完整性上，「個別資格標準」提示的表現優於「整體資格標準」提示。這顯示，相較於在單一查詢中請模型說明每一項個別資格標準的細節，當要求它在單一查詢中找出 SNAP 或 Medicaid 的所有資格標準時，LLM 的效果較差。

• Income Scenario prompts performed the worst in terms of being current and state-specific. The LLMs struggled to provide current and accurate income limits and compare them to the income limits in the prompts.

> • 在「現行且符合州別特定」這一項上，「收入情境」提示表現最差。LLM 很難提供現行且正確的收入上限，並與提示中的收入數字相比較。

Average Scores by Model

依模型的平均分數

Figure 3: Model Effectiveness

圖 3：模型成效

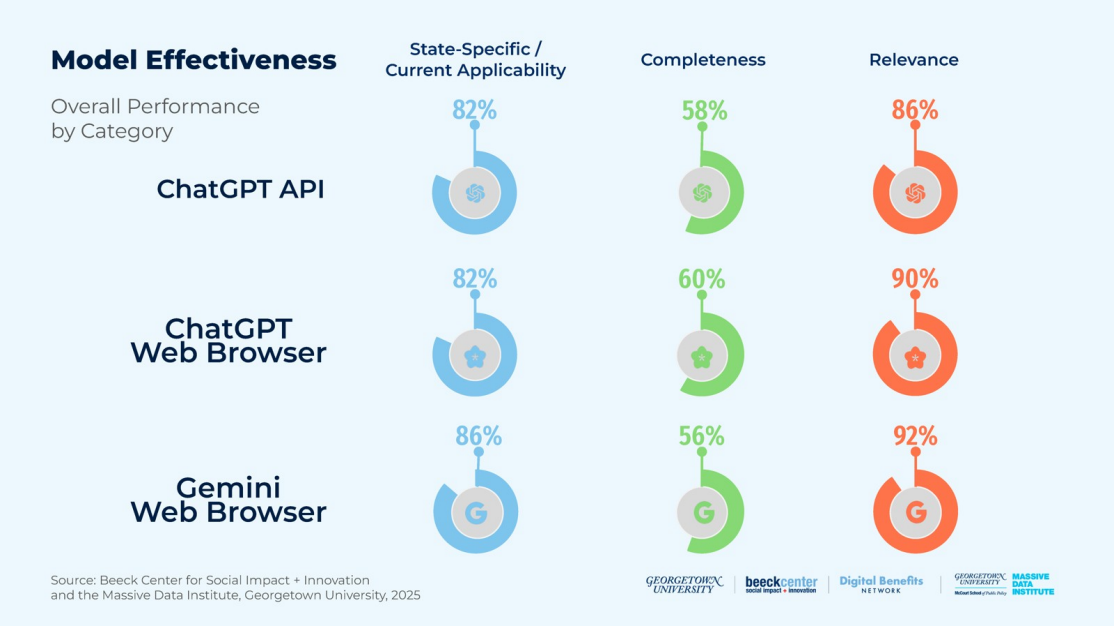


Table 4: Average Scores by Model

表 4：依模型的平均分數（表格內容保留英文原文）

	State-Specific/ Current Applicability		**Completeness**		**Relevance of Response**	
Model	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
ChatGPT API	4.1	82%	2.9	58%	4.3	86%
ChatGPT Web Browser	4.1	82%	3.0	60%	4.5	90%
Gemini Web Browser	4.3	86%	2.8	56%	4.6	92%

This table shows the average scores across both SNAP and Medicaid responses by the model used (i.e., GPT-4o API, ChatGPT web browser, or the Gemini web browser).

> 此表顯示依所用模型（即 GPT-4o API、ChatGPT 網頁瀏覽器或 Gemini 網頁瀏覽器）計算的 SNAP 與 Medicaid 回應平均分數。

Notable results in this table include:

> 此表中值得注意的結果包括：

• There were no major differences in the three models' performances. However, the Gemini web browser had the highest score for current/state-specific applicability and relevance, while the ChatGPT web browser had the highest score for completeness. The GPT-4o API had the lowest score for relevance, while the Gemini web browser had the lowest score for completeness. The GPT-4o API and ChatGPT web browser were tied for the lowest score for current/state-specific applicability.

> • 三個模型的表現沒有重大差異。不過，Gemini 網頁瀏覽器在現行／州別特定適用性與相關性上得分最高，ChatGPT 網頁瀏覽器則在完整性上得分最高。GPT-4o API 在相關性上得分最低，Gemini 網頁瀏覽器在完整性上得分最低。GPT-4o API 與 ChatGPT 網頁瀏覽器在現行／州別特定適用性上並列最低。

Average Scores by Program

依計畫的平均分數

Table 5: Average Scores by Program

表 5：依計畫的平均分數（表格內容保留英文原文）

	State-Specific/ Current Applicability		**Completeness**		**Relevance of Response**	
Program	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
SNAP	4.1	82%	2.8	56%	4.4	88%
Medicaid	4.2	84%	3.0	60%	4.5	90%

This table shows the average scores for both SNAP and Medicaid across all LLMs tested.

> 此表顯示所有受測 LLM 在 SNAP 與 Medicaid 上的平均分數。

Notable results in this table include:

> 此表中值得注意的結果包括：

• The average scores for Medicaid responses were slightly higher than for SNAP responses across all three rubric criteria.

> • 在三項評分標準上，Medicaid 回應的平均分數都略高於 SNAP 回應。

Gemini Web Browser Scores by Program

Gemini 網頁瀏覽器依計畫的分數

Table 6: Gemini Web Browser Scores by Program

表 6：Gemini 網頁瀏覽器依計畫的分數（表格內容保留英文原文）

	State-Specific/ Current Applicability		**Completeness**		**Relevance of Response**	
Program	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
SNAP	4.4	88%	2.7	54%	4.5	90%
Medicaid	4.2	84%	3.0	60%	4.6	92%

This table shows the average scores for both SNAP and Medicaid for only Gemini-generated responses.

> 此表僅顯示 Gemini 所生成回應在 SNAP 與 Medicaid 上的平均分數。

Notable results in this table include:

> 此表中值得注意的結果包括：

• For SNAP, Gemini outperformed the average score across all three models in terms of current and state-specific applicability and relevance, but underperformed in terms of completeness.

> • 在 SNAP 方面，Gemini 於現行／州別特定適用性與相關性上優於三個模型的平均分數，但在完整性上低於平均。

• For Medicaid, Gemini had an average performance in terms of current and state-specific applicability and completeness, but outperformed the average in terms of relevance.

> • 在 Medicaid 方面，Gemini 於現行／州別特定適用性與完整性上表現與平均相當，但在相關性上優於平均。

ChatGPT Web Browser Scores by Program

ChatGPT 網頁瀏覽器依計畫的分數

Table 7: ChatGPT Web Browser Scores by Program

表 7：ChatGPT 網頁瀏覽器依計畫的分數（表格內容保留英文原文）

	State-Specific/ Current Applicability		Completeness		Relevance of Response	
Program	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
SNAP	4.1	82%	3.0	60%	4.5	90%
Medicaid	4.1	82%	2.9	58%	4.4	88%

This table shows the average scores for both SNAP and Medicaid using only responses generated by the ChatGPT web browser.

> 此表僅使用 ChatGPT 網頁瀏覽器所生成的回應，顯示 SNAP 與 Medicaid 的平均分數。

Notable results in this table include:

> 此表中值得注意的結果包括：

• For SNAP, the ChatGPT web browser outperformed the average score across all three models in terms of completeness and relevance and had an average performance in terms of current and state-specific applicability.

> • 在 SNAP 方面，ChatGPT 網頁瀏覽器於完整性與相關性上優於三個模型的平均分數，於現行／州別特定適用性上表現與平均相當。

• For Medicaid, the ChatGPT web browser outperformed the average score in terms of current and state-specific applicability and underperformed in terms of completeness and relevance.

> • 在 Medicaid 方面，ChatGPT 網頁瀏覽器於現行／州別特定適用性上優於平均分數，於完整性與相關性上低於平均。

GPT-4o API Scores by Program

GPT-4o API 依計畫的分數

Table 8: GPT-4o API Scores by Program

表 8：GPT-4o API 依計畫的分數（表格內容保留英文原文）

	State-Specific/ Current Applicability		Completeness		Relevance of Response	
Program	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
SNAP	4.0	80%	2.7	54%	4.2	84%
Medicaid	4.3	86%	3.2	64%	4.4	88%

This table shows the average scores for SNAP and Medicaid, using only responses generated by the ChatGPT API.

> 此表僅使用 ChatGPT API 所生成的回應，顯示 SNAP 與 Medicaid 的平均分數。（按：原文此處寫 ChatGPT API，依上下文應指 GPT-4o API。）

Notable results in this table include:

> 此表中值得注意的結果包括：

- For SNAP, the GPT-4o API underperformed on the average score across all three models for all rubric criteria.

> • 在 SNAP 方面，GPT-4o API 在所有評分標準上都低於三個模型的平均分數。

- For Medicaid, the GPT-4o API outperformed the average score across all three models in terms of current and state-specific applicability and completeness, and underperformed in terms of relevance.

> • 在 Medicaid 方面，GPT-4o API 於現行／州別特定適用性與完整性上優於三個模型的平均分數，於相關性上低於平均。

Average Scores by State and Program

依州別與計畫的平均分數

Figure 4: Average Scores for State and Program

圖 4：各州與各計畫的平均分數

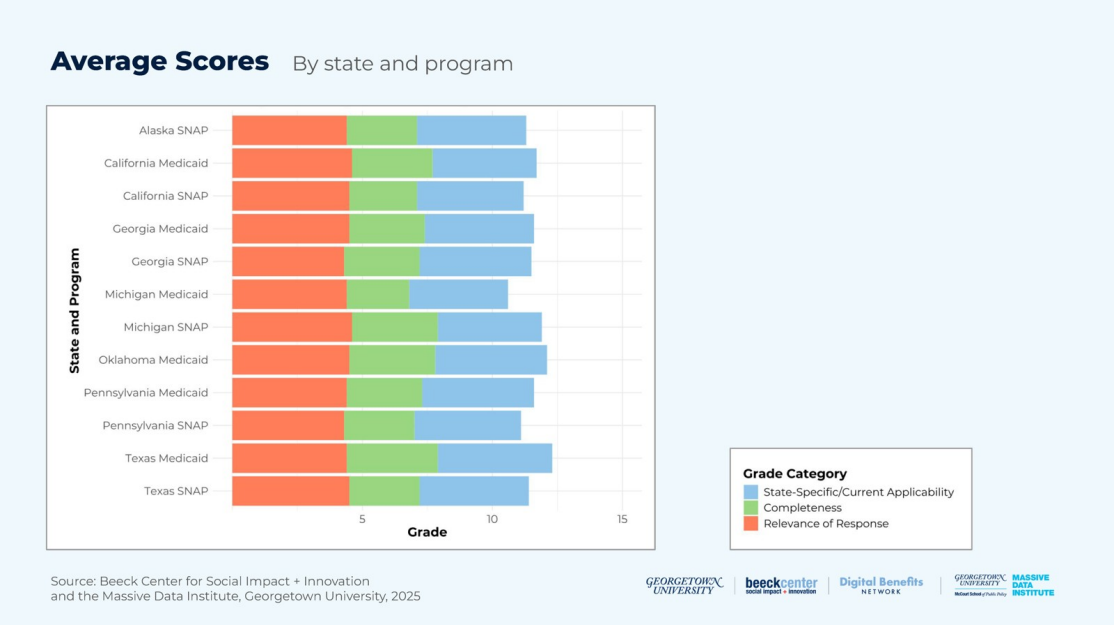


Table 9: Average Scores by State and Program

表 9：依州別與計畫的平均分數（表格內容保留英文原文）

	State-Specific/ Current Applicability		**Completeness**		**Relevance of Response**	
State and Program	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
Alaska SNAP	4.2	84%	2.7	54%	4.4	88%
California SNAP	4.1	82%	2.6	52%	4.5	90%
California Medicaid	4.0	80%	3.1	62%	4.6	92%
Georgia SNAP	4.3	86%	2.9	58%	4.3	86%
Georgia Medicaid	4.2	84%	2.9	58%	4.5	90%
Michigan SNAP	4.0	80%	3.3	66%	4.6	92%
Michigan Medicaid	3.8	76%	2.4	48%	4.4	88%
Oklahoma Medicaid	4.3	86%	3.3	66%	4.5	90%
Pennsylvania SNAP	4.1	81%	2.7	54%	4.3	86%
Pennsylvania Medicaid	4.3	86%	2.9	58%	4.4	88%
Texas SNAP	4.2	84%	2.7	54%	4.5	90%
Texas Medicaid	4.4	88%	3.5	70%	4.4	88%

This table shows the average scores across all three models for each state and program combination.

> 此表顯示三個模型合計下，各州與計畫組合的平均分數。

Notable results in this table include:

> 此表中值得注意的結果包括：

• Texas Medicaid scored highest in current and state-specific applicability and completeness, while California Medicaid and Michigan SNAP both tied for the highest relevance score.

> • 德州 Medicaid 在現行／州別特定適用性與完整性上得分最高，加州 Medicaid 與密西根州 SNAP 則在相關性上並列最高。

• Michigan Medicaid received the lowest scores for current and state-specific applicability and relevance, while Georgia SNAP and Pennsylvania SNAP tied for the lowest relevance score.

> • 密西根州 Medicaid 在現行／州別特定適用性與相關性上得分最低，而喬治亞州 SNAP 與賓夕法尼亞州 SNAP 在相關性上並列最低。

Gemini Web Browser Scores by State and Program

Gemini 網頁瀏覽器依州別與計畫的分數

Figure 5: Gemini Web Browser Scores for State and Program

圖 5：Gemini 網頁瀏覽器各州與各計畫的分數

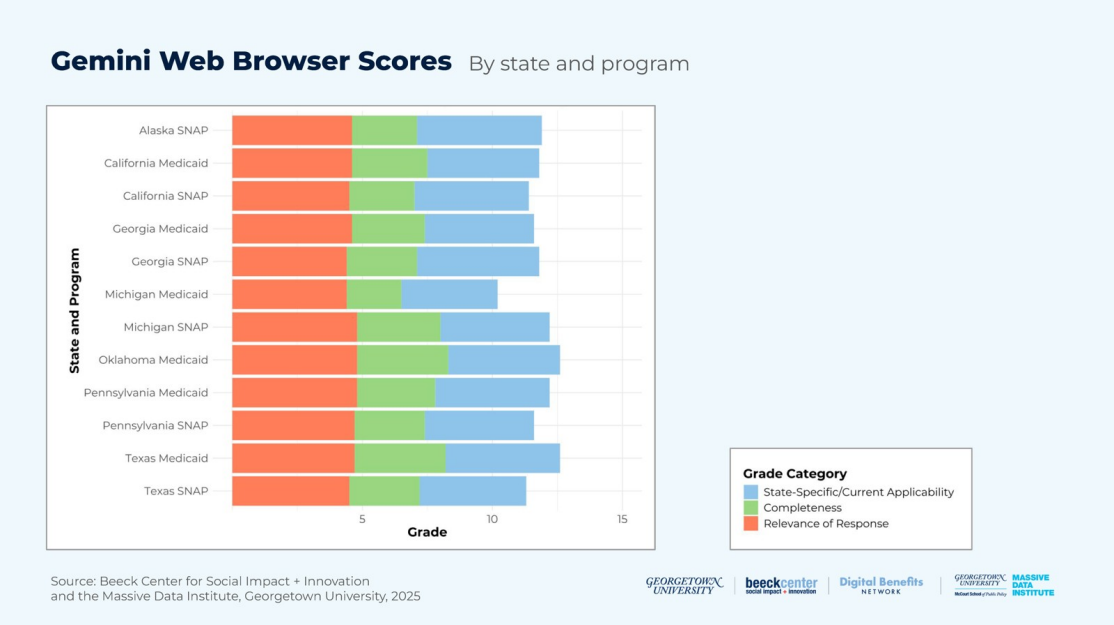


Table 10: Gemini Web Browser Scores by State and Program

表 10：Gemini 網頁瀏覽器依州別與計畫的分數（表格內容保留英文原文）

	State-Specific/ Current Applicability		**Completeness**		**Relevance of Response**	
State and Program	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
Alaska SNAP	4.8	96%	2.5	50%	4.6	92%
California SNAP	4.4	88%	2.5	50%	4.5	90%
California Medicaid	4.3	86%	2.9	58%	4.6	92%
Georgia SNAP	4.7	94%	2.7	54%	4.4	88%
Georgia Medicaid	4.2	84%	2.8	56%	4.6	92%
Michigan SNAP	4.2	84%	3.2	64%	4.8	96%
Michigan Medicaid	3.7	74%	2.1	42%	4.4	88%
Oklahoma Medicaid	4.3	86%	3.5	70%	4.8	98%
Pennsylvania SNAP	4.2	84%	2.7	54%	4.7	94%
Pennsylvania Medicaid	4.4	88%	3	60%	4.8	96%
Texas SNAP	4.1	81%	2.7	54%	4.5	90%
Texas Medicaid	4.4	88%	3.5	70%	4.7	94%

This table shows the average scores for each state and program combination for only Gemini-generated responses.

> 此表僅顯示 Gemini 所生成回應在各州與計畫組合上的平均分數。

Notable results in this table include:

> 此表中值得注意的結果包括：

- Alaska SNAP received the highest score for current and state-specific applicability. Oklahoma Medicaid and Texas Medicaid tied for the highest completeness score, while Michigan SNAP* and Pennsylvania Medicaid tied for the highest relevance score.

> • 阿拉斯加州 SNAP 在現行／州別特定適用性上得分最高。奧克拉荷馬州 Medicaid 與德州 Medicaid 在完整性上並列最高，密西根州 SNAP* 與賓夕法尼亞州 Medicaid 在相關性上並列最高。

- Michigan Medicaid* received the lowest scores for current and state-specific applicability and completeness and tied with Georgia SNAP for the lowest relevance score.

> • 密西根州 Medicaid* 在現行／州別特定適用性與完整性上得分最低，並在相關性上與喬治亞州 SNAP 並列最低。

*Please note that the Gemini responses for Michigan SNAP and Medicaid were generated at a later date than the responses for other programs.

> *請注意，Gemini 針對密西根州 SNAP 與 Medicaid 的回應，產生時間晚於其他計畫的回應。

ChatGPT Web Browser Scores by State and Program

ChatGPT 網頁瀏覽器依州別與計畫的分數

Figure 6: ChatGPT Web Browser Scores for State and Program

圖 6：ChatGPT 網頁瀏覽器各州與各計畫的分數

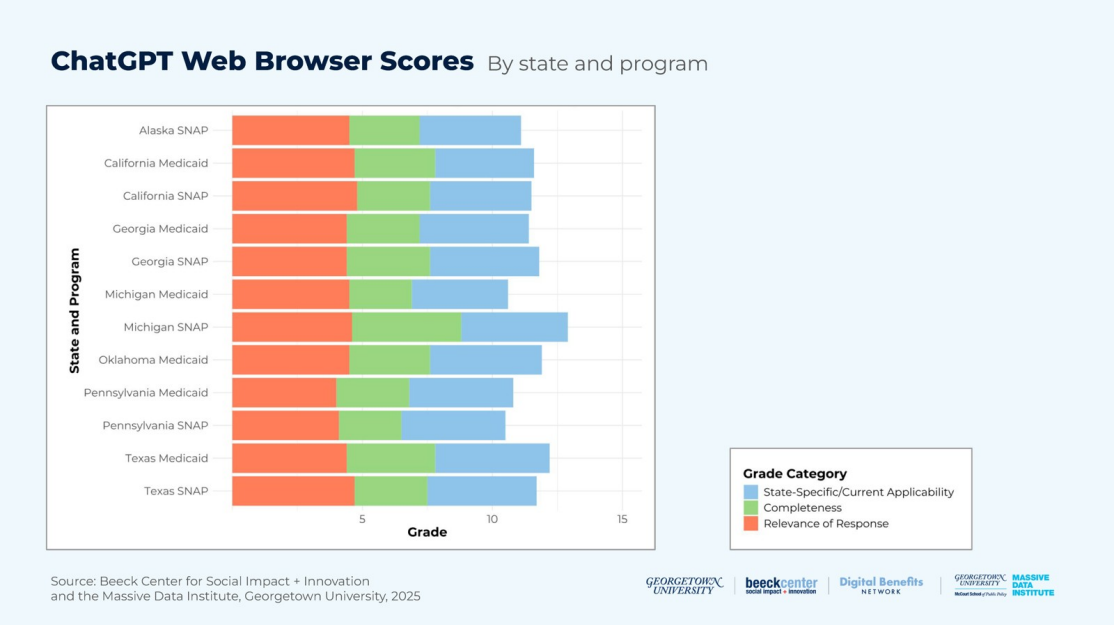


Table 11: ChatGPT Web Browser Scores by State and Program

表 11：ChatGPT 網頁瀏覽器依州別與計畫的分數（表格內容保留英文原文）

	State-Specific/ Current Applicability		**Completeness**		**Relevance of Response**	
State and Program	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
Alaska SNAP	3.9	78%	2.7	54%	4.5	90%
California SNAP	3.9	78%	2.8	56%	4.8	96%
California Medicaid	3.8	76%	3.1	62%	4.7	94%
Georgia SNAP	4.2	84%	3.2	64%	4.4	88%
Georgia Medicaid	4.2	84%	2.8	56%	4.4	88%
Michigan SNAP	4.1	82%	4.2	84%	4.6	92%
Michigan Medicaid	3.7	74%	2.4	48%	4.5	90%
Oklahoma Medicaid	4.3	86%	3.1	62%	4.5	90%
Pennsylvania SNAP	4.0	80%	2.4	48%	4.1	82%
Pennsylvania Medicaid	4.0	80%	2.8	56%	4.0	80%
Texas SNAP	4.2	84%	2.8	56%	4.7	94%
Texas Medicaid	4.4	88%	3.4	68%	4.4	88%

This table shows the average scores for each state and program combination for responses generated by the ChatGPT web browser only.

> 此表僅顯示 ChatGPT 網頁瀏覽器所生成回應在各州與計畫組合上的平均分數。

Notable results in this table include:

> 此表中值得注意的結果包括：

- Texas Medicaid received the highest score for current and state-specific applicability, while Michigan SNAP had the highest score for completeness, and California SNAP received the highest score for relevance.

> • 德州 Medicaid 在現行／州別特定適用性上得分最高，密西根州 SNAP 在完整性上得分最高，加州 SNAP 在相關性上得分最高。

- Michigan Medicaid had the lowest score for current and state-specific applicability and tied with Pennsylvania SNAP for the lowest score for completeness. Pennsylvania Medicaid had the lowest score for relevance.

> • 密西根州 Medicaid 在現行／州別特定適用性上得分最低，並在完整性上與賓夕法尼亞州 SNAP 並列最低。賓夕法尼亞州 Medicaid 在相關性上得分最低。

GPT-4o API Scores by State and Program

GPT-4o API 依州別與計畫的分數

Figure 7: GPT-4o API Scores for State and Program

圖 7：GPT-4o API 各州與各計畫的分數

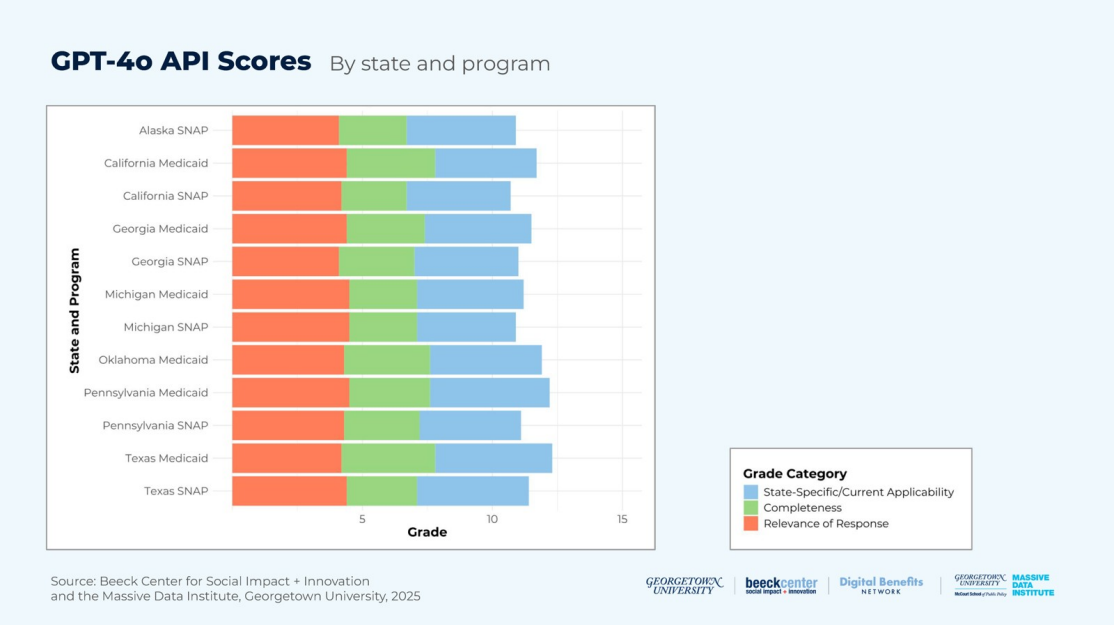


Table 12: GPT-4o API Scores by State and Program

表 12：GPT-4o API 依州別與計畫的分數（表格內容保留英文原文）

	State-Specific/ Current Applicability		**Completeness**		**Relevance of Response**	
State and Program	average performance score	average performance score in %	average performance score	average performance score in %	average performance score	average performance score in %
Alaska SNAP	4.2	84%	2.6	52%	4.1	82%
California SNAP	4	80%	2.5	50%	4.2	84%
California Medicaid	3.9	78%	3.4	68%	4.4	88%
Georgia SNAP	4.0	80%	2.9	58%	4.1	82%
Georgia Medicaid	4.1	82%	3	60%	4.4	88%
Michigan SNAP	3.8	76%	2.6	52%	4.5	90%
Michigan Medicaid	4.1	82%	2.6	52%	4.5	90%
Oklahoma Medicaid	4.3	86%	3.3	66%	4.3	86%
Pennsylvania SNAP	3.9	78%	2.9	58%	4.3	86%
Pennsylvania Medicaid	4.6	92%	3.1	62%	4.5	90%
Texas SNAP	4.3	86%	2.7	54%	4.4	88%
Texas Medicaid	4.5	90%	3.6	72%	4.2	84%

This table shows the average scores for each state and program combination for only GPT-4o API-generated responses.

> 此表僅顯示 GPT-4o API 所生成回應在各州與計畫組合上的平均分數。

Notable results in this table include:

> 此表中值得注意的結果包括：

• Pennsylvania Medicaid had the highest score for currency and state-specific applicability; Texas Medicaid had the highest score for completeness; and Michigan SNAP, Michigan Medicaid, and Pennsylvania Medicaid were tied for receiving the highest score for relevance.

> • 賓夕法尼亞州 Medicaid 在時效性與州別特定適用性上得分最高；德州 Medicaid 在完整性上得分最高；密西根州 SNAP、密西根州 Medicaid 與賓夕法尼亞州 Medicaid 在相關性上並列最高。

• Michigan SNAP had the lowest score for current and state-specific applicability, while California SNAP had the lowest score for completeness. Alaska SNAP and Georgia SNAP were tied for the lowest score for relevance.

> • 密西根州 SNAP 在現行／州別特定適用性上得分最低，加州 SNAP 在完整性上得分最低。阿拉斯加州 SNAP 與喬治亞州 SNAP 在相關性上並列最低。

Considerations for Public Benefits Use Cases

對公共福利使用情境的考量

While we applied these methods to specific states and policies, there are also considerations for wider public benefits use cases.

> 雖然我們把這些方法套用在特定的州與政策上，但對更廣泛的公共福利使用情境也有一些考量。

People seeking benefits information are likely receiving inaccurate information from chatbots.

> 尋求福利資訊的民眾，很可能正從聊天機器人得到不正確的資訊。

When generating responses based only on their training data or internet sources, the web browser versions of ChatGPT and Gemini, and the GPT-4o API, do not reliably generate current and state-specific, complete, and relevant responses to questions about SNAP and Medicaid eligibility policies.

> 當僅依其訓練資料或網路來源生成回應時，ChatGPT 與 Gemini 的網頁瀏覽器版本，以及 GPT-4o API，無法穩定地針對 SNAP 與 Medicaid 資格政策的問題，產出現行且符合州別、完整且相關的回應。

It is challenging to know where chatbots are sourcing information from.

> 很難知道聊天機器人的資訊從哪裡來。

While data sources used by models likely significantly influenced how responses' scored on our rubric, the lack of citations made it difficult to identify specific sources for most responses. The information provided in the responses likely came from official policy documents on government sites or third-party sources. Furthermore, we did not find a consistent pattern in our scores for the different state and program combinations or the different models, underscoring the challenge of relying only on the models' training data or other internet sources to gain accurate policy information. For this reason, in Experiment 2, we decided to assess how well the models answered questions when directly provided official policy documents.

> 雖然模型所用的資料來源很可能大幅影響回應在我們評分準則上的得分，但由於缺少引用，大多數回應都難以確認具體來源。回應中提供的資訊，可能來自政府網站上的官方政策文件，也可能來自第三方來源。此外，我們在不同州與計畫的組合或不同模型的分數上，並未發現一致的規律，這凸顯了僅依賴模型的訓練資料或其他網路來源來取得正確政策資訊的挑戰。因此，在實驗二中，我們決定評估當直接提供官方政策文件時，模型回答問題的表現如何。

Chatbots used confident, authoritative language to convey information, even when inaccurate.

> 聊天機器人即使在資訊不正確時，也使用有把握、權威的語氣傳達資訊。

When the models gave incorrect information, they often did so in a confident tone that would likely cause misunderstandings for readers who were not subject matter experts. Prior research shows that this is even the case for some models explicitly asked to quantify uncertainty9. This provides evidence for the necessity of a standardized method to quantify the uncertainty in AI responses, which is particularly important for Rules as Code generation where correct details are essential.

> 當模型給出不正確資訊時，往往是以有把握的口氣呈現，這很可能讓不是主題專家的讀者產生誤解。先前的研究顯示，即使是被明確要求量化不確定性的某些模型，也是如此[9]。這證明了有必要建立標準化的方法來量化 AI 回應中的不確定性，這對於「規則即程式碼」的生成特別重要，因為正確的細節是不可或缺的。

Experiment 2: Focusing LLMs on Specific Benefits Policy Documents

實驗二：讓 LLM 聚焦於特定的福利政策文件

Motivation and Question

動機與問題

For our second experiment, the goal was to determine how effectively LLMs summarized eligibility criteria from policy documents into plain language and formats easily convertible to pseudocode. A primary motivation for this study was to investigate how different types of questions affected the extraction of eligibility information from policies. For Experiment 2, we used Retrieval-Augmented Generation (RAG), a technique that allowed an LLM to use not only its training data to respond to queries, but also knowledge from authoritative sources

provided by the user. This experiment allowed us to test the impact of adding an authoritative policy document dataset to the process.

> 在第二項實驗中，目標是判斷 LLM 把政策文件中的資格標準摘要成白話文字、以及易於轉換成偽代碼的格式，效果有多好。這項研究的一個主要動機，是探討不同類型的問題如何影響從政策中擷取資格資訊。在實驗二中，我們使用檢索增強生成（RAG），這是一種技術，讓 LLM 回應查詢時，除了使用其訓練資料之外，還能利用使用者提供的權威來源知識。這項實驗讓我們能測試把權威政策文件資料集加進流程後的影響。

Overall, the main question guiding our study was:

> 整體而言，引導本研究的主要問題是：

- How well does the GPT-4o API generate accurate, complete, and logical summaries of benefits policy rules in response to different types of prompts?

> - GPT-4o API 面對不同類型的提示，生成福利政策規則摘要的準確性、完整性與邏輯性有多好？

Retrieval-Augmented Generation (RAG) is a process that combines a retrieval system with a generative model (e.g., an LLM like GPT-4 or Llama) to improve the quality and relevance of responses. RAG complements text generation with information from authoritative data sources, e.g. peer reviewed journal articles or actual state policies. By using RAG techniques, conversational interfaces use the relevant information from the authoritative data sources to augment and guide their response. In other words, RAG combines the generative power of a neural AI model with specialized information from a database to generate responses that are context-aware and generally more accurate. You can read more about RAG in this survey10.

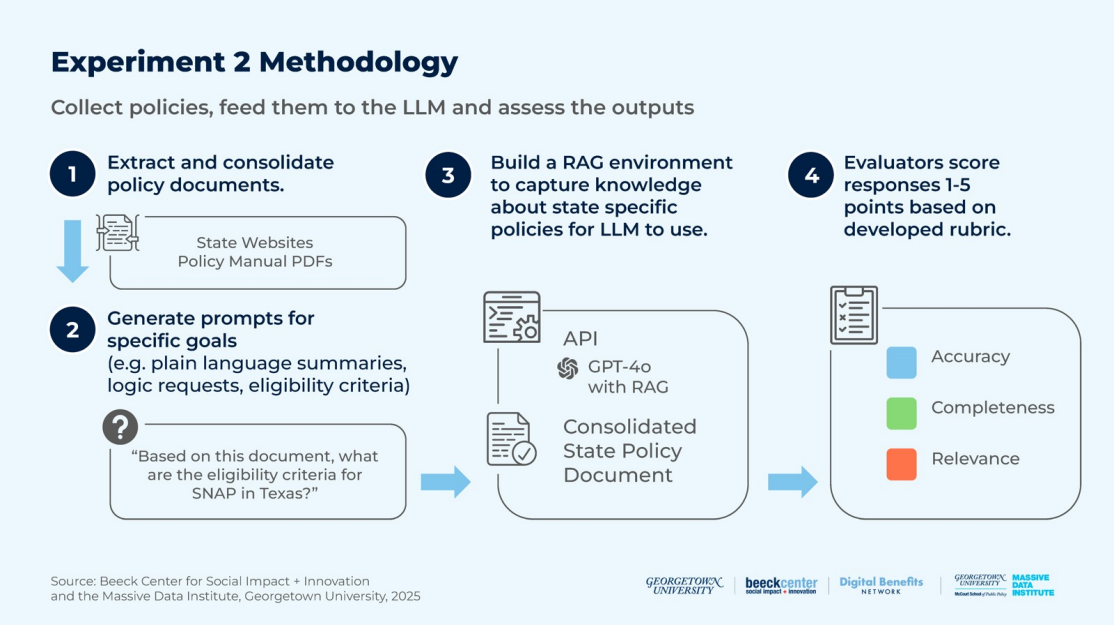
> 檢索增強生成（RAG）是一種把檢索系統與生成模型（例如 GPT-4 或 Llama 這類 LLM）結合的流程，用以提升回應的品質與相關性。RAG 以權威資料來源的資訊（例如經同儕審查的期刊文章或實際的州政策）來補強文字生成。運用 RAG 技術時，對話式介面會利用權威資料來源中的相關資訊來增強並引導其回應。換言之，RAG 把神經網路 AI 模型的生成能力，與資料庫中的專門資訊結合，產生能掌握情境、且通常更準確的回應。關於 RAG 的更多內容，可參閱這份綜述[10]。

Methodology

方法

Figure 8: Experiment 2 Methodology

圖 8：實驗二方法



1. Obtaining Data on Policies

1. 取得政策資料

Gather and Assess: We acquired policy documents, guides, and links by consolidating PDF documents and using web scraping.

> 蒐集與評估：我們整合 PDF 文件並使用網路爬取（web scraping），取得政策文件、指南與連結。

Evaluate Relevancy: We ensured the data gathered was relevant to the policies in question.

> 評估相關性：我們確保蒐集到的資料與所探討的政策相關。

Create a Single PDF: We consolidated documents into a single PDF for easier reference and analysis by state. For example, if a state had separate PDFs for each policy section, they were combined into one PDF.

> 建立單一 PDF：我們依州別把文件合併成單一 PDF，方便參照與分析。例如，若某州每個政策章節都有獨立的 PDF，就把它們合併成一份。

2. Prompt Generation

2. 產生提示

Create Specific Prompts: We tailored prompts for each policy and state, asking the LLM about only one policy at a time to ensure we received targeted responses. For example, the prompts for our SNAP eligibility analysis were divided into seven categories based on specific goals.

> 建立具體提示：我們針對每項政策與每個州量身設計提示，一次只向 LLM 詢問一項政策，以確保得到聚焦的回應。例如，我們的 SNAP 資格分析提示，依具體目標分為七個類別。

- “Please provide a plain language summary of the eligibility criteria for SNAP in Georgia found in the provided document.”

> - 「請就所提供文件中的喬治亞州 SNAP 資格標準，提供白話摘要。」

- “Please summarize the non-financial and financial eligibility requirements for SNAP in Georgia found in the provided document.”

> - 「請摘要所提供文件中的喬治亞州 SNAP 非財務與財務資格要求。」

- “Please do not skip any important details.”

> - 「請不要略過任何重要細節。」

- “Please keep the response to 500 words or less.”

> - 「請將回應控制在 500 字以內。」

- “Summarize the income, resource, work, or other requirements for SNAP.”

> - 「摘要 SNAP 的收入、資源、工作或其他要求。」

- “Please summarize the eligibility policy for SNAP in Texas found in the uploaded document.”

> - 「請摘要所上傳文件中的德州 SNAP 資格政策。」

- “Based on this document, please provide the logic for what makes someone eligible for SNAP in Georgia.”

> - 「請根據這份文件，提供什麼樣的人有資格領取喬治亞州 SNAP 的邏輯。」

- “Based on this document, what are the eligibility criteria for SNAP in Texas?”

> - 「根據這份文件，德州 SNAP 的資格標準是什麼？」

- “I am an unemployed single dad in Georgia. What determines my eligibility for SNAP?”

> - 「我是喬治亞州失業的單親爸爸。什麼會決定我是否符合 SNAP 資格？」

- “I am a pregnant woman in Georgia. What determines my eligibility for SNAP?”

> - 「我是喬治亞州的孕婦。什麼會決定我是否符合 SNAP 資格？」

Additionally, we experimented with a prompt for generating mermaid code—a syntax that uses text to create diagrams—to visualize eligibility logic. However, GPT-4o was unable to produce it.

> 此外，我們也嘗試了一個用來產生 mermaid 程式碼（一種用文字產生圖表的語法）的提示，以視覺化資格邏輯。然而，GPT-4o 無法產出。

Note: The example prompts above reference Georgia’s SNAP program, but were adapted for all states and programs included in the project.

> 附註：上述範例提示以喬治亞州的 SNAP 計畫為例，但已針對本專案納入的所有州與計畫加以調整。

3. Building a RAG Environment to Ask Questions about Specific State Policies

3. 建立 RAG 環境，以詢問特定州政策的問題

Setting up LangChain: We used the LangChain framework, which allows building more advanced AI applications by combining language models with other tools and complex workflows. We implemented LangChain in this step to facilitate the interaction and data processing between the language model and the different sources of information used in the experiment.

> 設定 LangChain：我們使用 LangChain 框架，它可以把語言模型與其他工具及複雜工作流程結合，打造更進階的 AI 應用。我們在這個步驟導入 LangChain，以便於語言模型與實驗中所用各種資訊來源之間的互動與資料處理。

LangChain has document loaders, allowing users to feed extracted policies into the LLM. Because LLMs can only process a certain amount of text at once, we split the text into manageable chunks by splitting character text. We also converted these chunks to embeddings by using the Embeddings class from Langchain. We used FAISS (Facebook AI Similarity Search) to perform an efficient similarity search between embeddings. After uploading the document, we also entered the prompts that we generated for each state and policy (as seen in the previous step).

> LangChain 具備文件載入器，讓使用者能把擷取出的政策餵給 LLM。由於 LLM 一次只能處理一定量的文字，我們以字元文字切分的方式，把文字切成容易處理的區塊（chunk）。我們也使用 LangChain 的 Embeddings 類別，把這些區塊轉成嵌入向量（embeddings）。我們使用 FAISS（Facebook AI Similarity Search）在嵌入向量之間進行有效率的相似度搜尋。上傳文件後，我們也輸入了為每個州與政策產生的提示（如前一步驟所示）。

4. Assessing LLM Outputs

4. 評估 LLM 的輸出

Evaluate Responses: For this experiment, our research group analyzed the responses generated by the LLM for accuracy, completeness and relevance, ensuring they met the criteria set for each policy.

> 評估回應：在這項實驗中，我們的研究小組針對 LLM 生成的回應，分析其準確性、完整性與相關性，確保它們符合為每項政策設定的標準。

Table 13: Experiment 2 Rubric

表 13：實驗二評分準則（表格內容保留英文原文）

Accuracy	**Is the information from the response accurate?**	**Score 1-5**
Completeness	Is the response thorough and does it cover all elements requested in the	Score 1-5

	prompt?	
Relevance	Is the response focused on the question, without adding irrelevant or unnecessary details?	Score 1-5

Read more about the Experiment 2 Rubric in the Appendix. →

> 關於實驗二評分準則的更多內容，請見附錄。→（譯按：準則三項各 1 至 5 分。「準確性」：資訊是否正確；「完整性」：回應是否周全、涵蓋提示要求的所有要素；「相關性」：回應是否聚焦於問題、沒有加入無關或不必要的細節。）

Developments and Challenges

發展與挑戰

As outlined in the methodology, we extracted SNAP policies for Alaska, California, Michigan, Georgia, Texas, and Pennsylvania, and Medicaid policies for Oklahoma, California, Georgia, Michigan, Pennsylvania, and Texas. Policy extraction excluded rule summaries on state websites, focusing only on current legislative documents or policy manuals for each state.

> 如方法所述，我們擷取了阿拉斯加州、加州、密西根州、喬治亞州、德州與賓夕法尼亞州的 SNAP 政策，以及奧克拉荷馬州、加州、喬治亞州、密西根州、賓夕法尼亞州與德州的 Medicaid 政策。政策擷取不含各州網站上的規則摘要，只聚焦於各州現行的立法文件或政策手冊。

A key challenge was policy fragmentation across separate PDFs. In Georgia's Online Directives Information System of the Department of Human Services (DHS), each section required a separate PDF download, while states like Pennsylvania and Alaska had interactive policy manuals that were easy to navigate on their websites but difficult to download and compile into a single document. For this process, we used Selenium to automate extraction and create a consolidated PDF.

> 一項關鍵挑戰是政策分散在不同的 PDF 中。在喬治亞州人類服務部（DHS）的線上指令資訊系統中，每個章節都需要另外下載一份 PDF；而賓夕法尼亞州與阿拉斯加州等州，則有互動式政策手冊，在網站上容易瀏覽，卻很難下載並彙整成單一文件。在這個過程中，我們使用 Selenium 自動化擷取，並建立彙整後的 PDF。

Another major challenge was scanned PDFs—like from California's State Medicaid Program Section 2—which prevented text selection, highlighting, and searching. This was especially problematic since LLMs rely on selectable text for reading and analysis.

> 另一項主要挑戰是掃描版 PDF（例如加州州 Medicaid 計畫第 2 節），這使得文字無法選取、標示與搜尋。這特別有問題，因為 LLM 仰賴可選取的文字來閱讀與分析。

It is important to note that we extracted policy manuals as-is, without edits, to evaluate LLM performance in generating accurate, complete, and logical summaries of the benefit rules.

> 值得注意的是，我們是照原樣擷取政策手冊、未經編修，以評估 LLM 在生成福利規則的準確、完整且合乎邏輯的摘要上的表現。

The tables below provide an overview of key observations from our research and extraction of SNAP and Medicaid policy manuals.

> 下列表格概述我們在研究與擷取 SNAP 及 Medicaid 政策手冊過程中的重要觀察。

Table 14: Observations of SNAP Manuals: Ease of Use and Accessibility for LLM Input

表 14：SNAP 手冊的觀察：作為 LLM 輸入的易用性與可取得性（表格內容保留英文原文）

State	Strengths	Challenges	Pages	Link
Alaska	Easy online accessibility, with a functional index and chapter navigation	Hard to obtain the full policy text directly; requires manual clicking or web scraping through sections, and some appendices are not readily accessible online.	350 pages, excluding transmittals	Alaska SNAP Manual
California	Website includes updated dates of files	Lengthy, making it difficult for AI tools like RAG to process in full.	1,395 pages	Eligibility and Assistance Standards (EAS) Manual
Michigan	A comprehensive guide that's available in one PDF format	Focuses on Medicaid and SEBT (Summer Electronic Benefit Transfer Program), in addition to SNAP Embedded TOC (Table of Contents) lacks organization and detail compared to the on-page TOC	1,214 pages (Same PDF as Michigan Medicaid).	Michigan Bridges Eligibility Policy Manuals
Georgia	Well-documented, easy to understand, with separate chapters and sections	Lacks hyperlinks to other sections. The manual is split across multiple PDFs, requiring manual merging for a complete view.	495 pages	Online Directives Information System of the Georgia Department of Human Services (DHS)
Texas	Accessible and easy to navigate, both online and in PDF form	Covers multiple benefit programs, not just SNAP	985 pages	Texas Works Handbook
Pennsylvania	Includes scenarios	The separate	553 pages	Pennsylvania

	and examples to make eligibility clear. Separate chapters and sections	sections can make it hard to obtain the full policy text directly; requires manual clicking or web scraping through sections		Supplemental Nutrition Assistance Program (SNAP) Handbook

Table 15: Observations on Medicaid Manuals: Ease of Use and Accessibility for LLM Input

表 15：Medicaid 手冊的觀察：作為 LLM 輸入的易用性與可取得性（表格內容保留英文原文）

State	Strengths	Challenges	Pages	Link
Oklahoma	Single PDF; no need to extract different things	Embedded Table of Contents (TOC) lacks organization and detail compared to the on-page TOC	1,368 pages	State Plan 05.30.2024.pdf Oklahoma Administrative Code
California	The website provides a table that organizes the manual's sections and includes its attachment	Scanned pages are not very searchable, revoked policy is sometimes crossed out with a pen mark	126 pages	California's State Medicaid Program (Section 2 – Coverage and Eligibility)
Michigan	A comprehensive guide that's available in one PDF format	Focuses on SNAP and SEBT (Summer Electronic Benefit Transfer Program) in addition to Medicaid Embedded Table of Contents (TOC) lacks organization and detail compared to the on-page TOC	1,227 pages	Michigan Bridges Eligibility Policy Manuals
Georgia	After merging all the PDFs, the manual is well-documented, easy to understand, and organized with distinct chapters and sections	Website with different sections, but requires manual extraction of each PDF from each section.	1,193 pages	Georgia Division of Family and Children Services Medicaid Policy Manual
Texas	A comprehensive guide that's available in PDF and easily accessible in HTML and as a website.	Important information about eligibility, such as A-1200 (Resources) and A-1300 (Income), is not included in the guide, but is linked to another webpage for reference. Focuses more on the	41 pages	Texas Works Handbook

		coverage of each program.		
Pennsylvania	Easy online accessibility, with a functional index and chapter navigation	The separate sections can make it hard to obtain the full policy text directly; requires manual clicking or web scraping through sections	942 pages	Pennsylvania Medical Assistance Eligibility Handbook

Findings

發現

After feeding the extracted policies into the ChatGPT-4o API using the RAG technique, we applied the specific prompts designed for each policy and state (step 4 of the methodology).

> 我們以 RAG 技術把擷取出的政策餵進 ChatGPT-4o API 之後，套用了為每項政策與每個州設計的具體提示（方法的第 4 步）。

For each state policy, the LLM generated responses to 19 prompts. Since we worked with six states for SNAP and six for Medicaid, and we used 19 prompts for each of these cases, this resulted in 228 responses. Our prompts, responses, and scores are available in the Experiment 2 Materials.

> 針對每個州的政策，LLM 對 19 個提示產生了回應。由於 SNAP 與 Medicaid 各處理六個州，且每種情況使用 19 個提示，總共產生 228 則回應。我們的提示、回應與分數收錄於「實驗二資料」。

The responses generated by the LLM were reviewed using the evaluation rubric mentioned above in step 5 of the methodology.

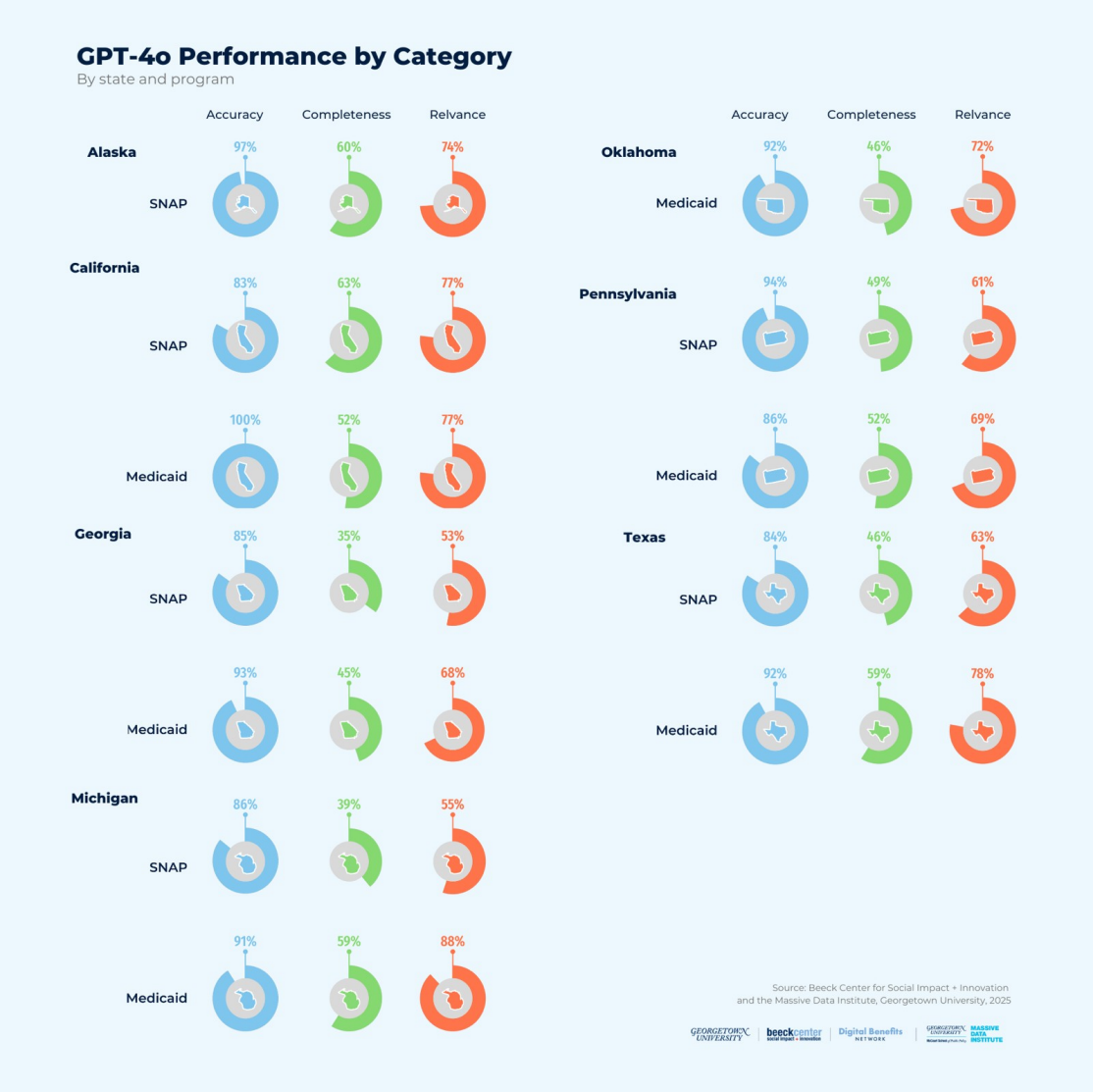
> LLM 所生成的回應，依上述方法第 5 步提到的評分準則進行審閱。

Our findings for how responses performed across three rubric categories— accuracy, completeness, and relevance of response—are included in this spreadsheet. They include both “average rubric performance scores” and “average percentage performance scores,” which convert the average rubric score into a percentage, for standardized comparison.

> 三個評分類別（準確性、完整性、回應相關性）上的回應表現發現，收錄於這份試算表中。其中同時包含「平均評分表現分數」，以及把平均評分轉成百分比、以便標準化比較的「平均表現百分比分數」。

Figure 9: GPT-4o API Performance by Category by State and Program

圖 9：GPT-4o API 依州別與計畫、各類別的表現



SNAP Results

SNAP 結果

When given prompts for SNAP, we found that the GPT-4o API generally returns accurate results; however, scores drop off significantly for completeness and relevance. This means that while the response may be accurate, it is likely missing information that would help inform an action or decision about SNAP. The response may also include information that is not relevant to the prompt, making it less useful for assessing eligibility.

> 面對 SNAP 的提示，我們發現 GPT-4o API 通常會回傳準確的結果；然而，完整性與相關性的分數明顯下滑。這表示，雖然回應可能是準確的，卻很可能遺漏了有助於針對 SNAP 採取行動或做出決定的資訊。回應也可能包含與提示無關的資訊，使它在評估資格時較不實用。

The tables below analyze SNAP guideline performance in six states (Alaska, California, Georgia, Michigan, Pennsylvania, and Texas) across three dimensions: accuracy, completeness, and relevance.

> 下列表格分析六個州（阿拉斯加、加州、喬治亞、密西根、賓夕法尼亞與德州）的 SNAP 指南表現，涵蓋三個面向：準確性、完整性與相關性。

Table 16: Average Performance of SNAP Policies in Accuracy, Completeness, and Relevance by States

表 16：各州 SNAP 政策在準確性、完整性與相關性的平均表現（表格內容保留英文原文）

	Accuracy	**Accuracy**	**Completeness**	**Completeness**	**Relevance**	**Relevance**
State	Average Performance Score	Average Performance Score by Percentage	Average Performance Score	Average Performance Score by Percentage	Average Performance Score	Average Performance Score by Percentage
Alaska	4.84	96.84%	3.00	60%	3.68	73.68%
California	4.16	83.16%	3.16	63.16%	3.84	76.84%
Georgia	4.26	85.26%	1.74	34.74%	2.63	52.63%
Michigan	4.32	86.32%	1.95	38.95%	2.74	54.74%
Pennsylvania	4.68	93.68%	2.47	49.47%	3.05	61.05%
Texas	4.21	84.21%	2.32	46.32%	3.16	63.16%

The table shows the overall performance level, considering all scores for a more comprehensive view of average response performance. The "average performance score" is a numerical evaluation on a scale of 1 to 5 based on the rubric. The "average performance score by percentage" converts this score into a percentage, providing a standardized way to compare results across different contexts.

> 此表顯示整體表現水準，並納入所有分數，以更全面地呈現回應的平均表現。「平均表現分數」是依評分準則給予的 1 至 5 分數值評估。「平均表現百分比分數」則把這個分數轉換成百分比，以提供在不同情境間標準化比較結果的方式。

Notable results in this table include:

> 此表中值得注意的結果包括：

• All states scored 4 or higher in evaluating response accuracy. Alaska scored the highest with an average performance score of 4.84 (96.84%). This was followed by Pennsylvania with 4.68 (93.68%) and Michigan with 4.32 (86.32%).

> • 所有州在評估回應準確性時都達到 4 分以上。阿拉斯加州最高，平均表現分數為 4.84（96.84%），其次是賓夕法尼亞州的 4.68（93.68%）與密西根州的 4.32（86.32%）。

• All states scored below 3.2 in evaluating response completeness. California had the highest completeness score with 3.16 (63.16%), followed by Alaska with a 3 (60%). Pennsylvania had an average score of 2.47 (49.47%), followed by Texas with 2.32 (46.32%). Michigan scored 1.95 (38.95%), and Georgia had the lowest score at 1.74 (34.74%).

• 所有州在評估回應完整性時都低於 3.2 分。加州的完整性分數最高，為 3.16（63.16%），其次是阿拉斯加州的 3（60%）。賓夕法尼亞州平均 2.47（49.47%），其次是德州的 2.32（46.32%）。密西根州得 1.95（38.95%），喬治亞州分數最低，為 1.74（34.74%）。

• All states scored between 2.5 to 3.8 in evaluating relevance of responses. California received the highest average relevance score of 3.84 (76.84%), followed by Alaska with an average score of 3.68 (73.68%). Texas received an average performance score of 3.16 (63.16%), followed by Pennsylvania at 3.05 (61.05%). Michigan and Georgia scored 2.74 (54.74%) and 2.63 (52.63%), respectively.

• 所有州在評估回應相關性時都落在 2.5 至 3.8 分之間。加州的平均相關性分數最高，為 3.84（76.84%），其次是阿拉斯加州的平均 3.68（73.68%）。德州的平均表現分數為 3.16（63.16%），其次是賓夕法尼亞州的 3.05（61.05%）。密西根州與喬治亞州分別為 2.74（54.74%）與 2.63（52.63%）。

Table 17: Percentage of Responses Receiving Top Scores for Accuracy, Completeness, and Relevance in SNAP

表 17：SNAP 中準確性、完整性與相關性獲得高分的回應百分比（表格內容保留英文原文）

	Accuracy	**Completeness**	**Relevance**
State	Percentage of responses that received a 4 or 5	Percentage of responses that received a 4 or 5	Percentage of responses that received a 4 or 5
Alaska	100%	26.32%	42.11%
California	84.21%	42.11%	57.89%
Georgia	73.68%	10.53%	26.32%
Michigan	78.95%	0%	31.58%
Pennsylvania	94.74%	21.05%	31.58%
Texas	78.95%	10.53%	36.84%

The table shows the percentage of all responses that received the highest scores (a 4 or 5) on accuracy, completeness, and relevance in SNAP.

此表顯示在 SNAP 中，準確性、完整性與相關性獲得最高分（4 或 5 分）的回應占全部回應的百分比。

Notable results in this table include:

此表中值得注意的結果包括：

• All states had more than 70% of responses rated as high in accuracy. Alaska performed best among all states, with 100% of responses rated 4 or 5, classified as very good or excellent for accuracy. This was followed by Pennsylvania, where 94.74% of responses received high scores, and California, with 84.21%; then Texas, Michigan and Georgia at 78.95%, 78.95% and 73.7%, respectively.

> • 所有州都有超過 70% 的回應在準確性上被評為高分。阿拉斯加州表現最好，100% 的回應獲得 4 或 5 分，在準確性上被歸類為非常好或優秀。其次是賓夕法尼亞州，94.74% 的回應獲得高分，加州為 84.21%；接著是德州、密西根州與喬治亞州，分別為 78.95%、78.95% 與 73.7%。

• All states had fewer than 43% of responses rated as high-scoring for completeness. California had the highest percentage, with only 42.11% of responses rated as good or excellent, followed by Alaska, Pennsylvania, Texas and Georgia with 26.32%, 21.05%, 10.53% and 10.53%, respectively. Michigan did not have any responses which scored 4 or 5, resulting in 0%.

> • 所有州在完整性上被評為高分的回應都少於 43%。加州的比例最高，也只有 42.11% 的回應被評為良好或優秀，其次依序是阿拉斯加州、賓夕法尼亞州、德州與喬治亞州，分別為 26.32%、21.05%、10.53% 與 10.53%。密西根州沒有任何回應得到 4 或 5 分，比例為 0%。

• All states had between 26% and 58% of responses rated as high-scoring for relevance. California had the highest percentage, with 57.89% of responses scoring 4 or 5, followed by Alaska with 42.1%, Texas with 36.84%, Pennsylvania with 31.58%, and Michigan with 31.58%. The state with the lowest percentage of high-scoring responses was Georgia at 26.32%.

> • 所有州在相關性上被評為高分的回應介於 26% 至 58% 之間。加州的比例最高，57.89% 的回應獲得 4 或 5 分，其次是阿拉斯加州的 42.1%、德州的 36.84%、賓夕法尼亞州的 31.58%，以及密西根州的 31.58%。高分回應比例最低的州是喬治亞州，為 26.32%。

Medicaid Results

Medicaid 結果

When given prompts for Medicaid, we found that the GPT-4o API generally returns accurate results; however, scores drop off significantly for completeness and relevance. This means that while the response may be accurate, it is likely missing information that would help inform an action or decision about Medicaid. The response may also include information that is not relevant to the prompt, making it less useful for assessing eligibility.

> 面對 Medicaid 的提示，我們發現 GPT-4o API 通常會回傳準確的結果；然而，完整性與相關性的分數明顯下滑。這表示，雖然回應可能是準確的，卻很可能遺漏了有助於針對 Medicaid 採取行動或做出決定的資訊。回應也可能包含與提示無關的資訊，使它在評估資格時較不實用。

The following tables analyzed the performance of Medicaid guidelines in six states (California, Georgia, Michigan, Oklahoma, Pennsylvania, and Texas) by evaluating three dimensions: accuracy, completeness, and relevance.

> 下列表格分析六個州（加州、喬治亞、密西根、奧克拉荷馬、賓夕法尼亞與德州）的 Medicaid 指南表現，評估三個面向：準確性、完整性與相關性。

Table 18: Average Performance in Accuracy, Completeness, and Relevance of Medicaid Policies by States

表 18：各州 Medicaid 政策在準確性、完整性與相關性的平均表現（表格內容保留英文原文）

	Accuracy	Accuracy	Completeness	Completeness	Relevance	Relevance
State	Average Performance Score	Average Performance Score by Percentage	Average Performance Score	Average Performance Score by Percentage	Average Performance Score	Average Performance score by Percentage
California	5.00	100%	2.58	51.58%	3.84	76.84%
Georgia	4.63	92.63%	2.26	45.26%	3.42	68.42%
Michigan	4.53	90.53%	2.95	58.95%	4.42	88.42%
Oklahoma	4.58	91.58%	2.32	46.32%	3.58	71.58%
Pennsylvania	4.32	86.32%	2.58	51.58%	3.47	69.47%
Texas	4.58	91.58%	2.95	58.95%	3.89	77.89%

The table shows the overall performance level of responses about Medicaid, including all scores, giving a more complete picture of how responses performed on average. The "average performance score" is a numerical evaluation on a scale of 1 to 5 based on the rubric. The "average performance score in %" converts this 1-5 score into a percentage, allowing for a more standardized comparison of results across settings.

> 此表顯示 Medicaid 回應的整體表現水準，納入所有分數，更完整地呈現回應的平均表現。「平均表現分數」是依評分準則給予的 1 至 5 分數值評估。「平均表現百分比分數」把這個 1 至 5 分轉成百分比，以便在不同設定間做更標準化的比較。

Notable results in this table include:

> 此表中值得注意的結果包括：

• All states scored 4 or higher in evaluating accuracy of responses. California scored the highest with an average performance score of 5.00 or 100%. This was followed by Georgia, Texas and Oklahoma with 4.6 (92.63%), 4.58 (91.58%) and 4.58 (91.58%) respectively, while Pennsylvania scores the lowest at 4.32 (86.32%), though still high.

> • 所有州在評估回應準確性時都達到 4 分以上。加州最高，平均表現分數為 5.00，即 100%。其次是喬治亞州、德州與奧克拉荷馬州，分別為 4.6（92.63%）、4.58（91.58%）與 4.58（91.58%），賓夕法尼亞州最低，為 4.32（86.32%），但仍屬偏高。

• All states scored below 3 in evaluating completeness of responses. Texas and Michigan hold the highest score, both with a 2.95 (or 58.95%), followed by Pennsylvania at 2.58 (51.58%), and California at 2.58 (51.58%). Oklahoma and Georgia show the lowest scores in this category with 2.32 (46.32%) and 2.26 (45.26%) respectively.

> ・所有州在評估回應完整性時都低於 3 分。德州與密西根州分數最高，皆為 2.95（即 58.95%），其次是賓夕法尼亞州的 2.58（51.58%）與加州的 2.58（51.58%）。奧克拉荷馬州與喬治亞州在此類別分數最低，分別為 2.32（46.32%）與 2.26（45.26%）。

• All states scored between 3.5 to 4.5 in evaluating relevance of responses. Michigan leads with a score of 4.42 (88.42%), while Texas (3.89, 77.89%) and Georgia (3.84, 76.84%) also perform well. Other states, such as Oklahoma and Pennsylvania, show similar and satisfactory results, with scores of 71.58% and 69.47%. California shows the lowest score with 3.47 (63.16%).

> ・所有州在評估回應相關性時都落在 3.5 至 4.5 分之間。密西根州以 4.42（88.42%）領先，德州（3.89，77.89%）與喬治亞州（3.84，76.84%）表現也不錯。其他州，如奧克拉荷馬州與賓夕法尼亞州，結果相近且令人滿意，分別為 71.58% 與 69.47%。加州分數最低，為 3.47（63.16%）。

• In general, the answers tend to be precise and relevant, but lack completeness.

> ・整體而言，答案往往精確且相關，但缺乏完整性。

Table 19: Percentage of Responses Receiving Top Scores for Accuracy, Completeness, and Relevance in Medicaid

表 19：Medicaid 中準確性、完整性與相關性獲得高分的回應百分比（表格內容保留英文原文）

	Accuracy	Completeness	Relevance
State	Percentage of responses that received a 4 or 5	Percentage of responses that received a 4 or 5	Percentage of responses that received a 4 or 5
California	100%	26.32%	52.63%
Georgia	94.74%	15.79%	52.63%
Michigan	84.21%	21.05%	78.95%
Oklahoma	89.47%	21.05%	52.63%
Pennsylvania	94.74%	10.53%	47.37%
Texas	94.74%	26.32%	63.16%

The table shows the distribution of top scores, highlighting how many responses were excellent or very good, meaning they received scores of 4 or 5.

> 此表顯示高分的分布情形，凸顯有多少回應屬於優秀或非常好，也就是獲得 4 或 5 分。

Notable results in this table include:

> 此表中值得注意的結果包括：

• All states had more than 84% of responses rated as high-scoring in terms of accuracy. California performed the best, with 100% of responses rated 4 or 5, classified as very good or excellent for accuracy. They were followed by Georgia, Pennsylvania, and Texas, with 94.74% for all.

> • 所有州都有超過 84% 的回應在準確性上被評為高分。加州表現最好，100% 的回應獲得 4 或 5 分，在準確性上被歸類為非常好或優秀。其次是喬治亞州、賓夕法尼亞州與德州，皆為 94.74%。

• All states had less than 27% of responses rated as high-scoring in terms of completeness. Texas and California had the highest percentage, with only 26.32% of responses rated as good or excellent, followed by Michigan and Oklahoma, both with 21.05%. Georgia and Pennsylvania had less than 16% of responses rated 4 or 5, with 15.79%, and 10.53%, respectively.

> • 所有州在完整性上被評為高分的回應都少於 27%。德州與加州的比例最高，也只有 26.32% 的回應被評為良好或優秀，其次是密西根州與奧克拉荷馬州，皆為 21.05%。喬治亞州與賓夕法尼亞州獲得 4 或 5 分的回應都不到 16%，分別為 15.79% 與 10.53%。

• All states had between 45% and 80% of responses rated as high-scoring in terms of relevance. Michigan led with 78.95%, standing out as the only state with more than 70% of responses scoring 4 or 5. Texas followed with 63.16%. The remaining states, including Oklahoma, California, Georgia, and Pennsylvania, each had less than 53% of responses rated 4 or 5, scoring 52.63%, 52.63%, 52.63%, and 47.37%, respectively.

> • 所有州在相關性上被評為高分的回應介於 45% 至 80% 之間。密西根州以 78.95% 領先，是唯一超過 70% 回應獲得 4 或 5 分的州。德州以 63.16% 居次。其餘各州，包括奧克拉荷馬州、加州、喬治亞州與賓夕法尼亞州，獲得 4 或 5 分的回應皆低於 53%，分別為 52.63%、52.63%、52.63% 與 47.37%。

Accuracy, Relevance, and Completeness Trends for Georgia SNAP and Oklahoma Medicaid

喬治亞州 SNAP 與奧克拉荷馬州 Medicaid 的準確性、相關性與完整性趨勢

Accuracy

準確性

• The AI-generated responses scored very high in accuracy, due to the fact that a document with reliable, up-to-date, and state-specific data was fed into the LLM. In the category of accuracy, 73.68% of Georgia SNAP LLM-generated responses performed well, scoring a 4 or 5 when evaluated with our rubric. For Oklahoma Medicaid, 89.47% of responses were high-achieving.

> • AI 生成的回應在準確性上得分很高，原因是餵給 LLM 的文件含有可靠、最新且屬於該州的資料。在準確性方面，喬治亞州 SNAP 的 LLM 生成回應中，有 73.68% 表現良好，依我們的評分準則獲得 4 或 5 分。奧克拉荷馬州 Medicaid 則有 89.47% 的回應屬於高分。

• Common accuracy errors included misinterpretation of eligibility criteria (e.g., age or work requirements), confusion between different groups' criteria, or an overemphasis on recent policies while overlooking older ones that still applied.

> • 常見的準確性錯誤包括：誤解資格標準（例如年齡或工作要求）、混淆不同族群的標準，或過度強調近期政策而忽略仍然適用的舊政策。

Completeness

完整性

• Fewer than a quarter of the LLM-generated responses from both programs scored very high in completeness. Only 10.53% of Georgia SNAP LLM-generated responses scored a 4 or 5 in completeness , while Oklahoma Medicaid saw slightly better results at 21.05%.

> • 兩項計畫中，LLM 生成回應在完整性上得到很高分的都不到四分之一。喬治亞州 SNAP 的 LLM 生成回應中，只有 10.53% 在完整性上得到 4 或 5 分，奧克拉荷馬州 Medicaid 的結果稍好，為 21.05%。

• Common completeness errors included missing crucial details necessary for understanding eligibility and a focus on irrelevant administrative information (e.g., application timelines, identity verification) instead of general eligibility criteria.

> • 常見的完整性錯誤包括：遺漏理解資格所必需的關鍵細節，以及把重點放在無關的行政資訊（例如申請時程、身分驗證），而不是一般的資格標準。

Relevance

相關性

• Only between 26% and 53% of LLM-generated responses for Georgia SNAP and Oklahoma Medicaid scored very highly for relevance. In relevance, 26.32% of Georgia SNAP AI-generated responses performed well, scoring a 4 or 5. For Oklahoma Medicaid, 52.63% of responses were high-achieving ones.

> • 喬治亞州 SNAP 與奧克拉荷馬州 Medicaid 的 LLM 生成回應中，在相關性上得到很高分的只占 26% 至 53%。在相關性方面，喬治亞州 SNAP 的 AI 生成回應有 26.32% 表現良好，獲得 4 或 5 分。奧克拉荷馬州 Medicaid 則有 52.63% 的回應屬於高分。

• Common relevance errors included off-topic answers or irrelevant information that included irrelevant elements that confused or diluted the main point.

> • 常見的相關性錯誤包括：答非所問，或包含無關資訊，其中的無關元素使人混淆或稀釋了主要重點。

Comparing Prompt Effectiveness for SNAP and Medicaid Across States

比較各州 SNAP 與 Medicaid 的提示成效

The following charts compare AI-generated responses with RAG for SNAP and Medicaid in Georgia, Pennsylvania, Michigan, and Texas. Each state contains up to 19 bars representing AI responses about a policy, and each bar corresponds to one of 19 questions categorized into six different question types:

> 下列圖表比較了喬治亞州、賓夕法尼亞州、密西根州與德州的 SNAP 與 Medicaid，使用 RAG 的 AI 生成回應。每個州最多有 19 根長條，代表 AI 對某項政策的回應，每根長條對應 19 個問題之一，這些問題分為六種問題類型：

- Plain Language Summary Request

> ・ 白話摘要請求

- General Summary Request

> ・ 一般摘要請求

- Logic Request and Logic in a Flowchart Requests

> ・ 邏輯請求與流程圖中的邏輯請求

- Eligibility Criteria (“Who is eligible?” or ”What are the eligibility criteria?”)

> ・ 資格標準（「誰有資格？」或「資格標準是什麼？」）

- Scenario-Based Prompts (“What would determine if I am eligible?”)

> ・ 情境式提示（「什麼會決定我是否符合資格？」）

These visualizations highlight how the LLM performed on different question types, with some questions scoring higher than others. Scores were broken down into three evaluation categories: accuracy, completeness, and relevance. It is important to note that question categories excluded from the charts received “N/A” results, indicating that there was no applicable answer for questions in that specific question category.

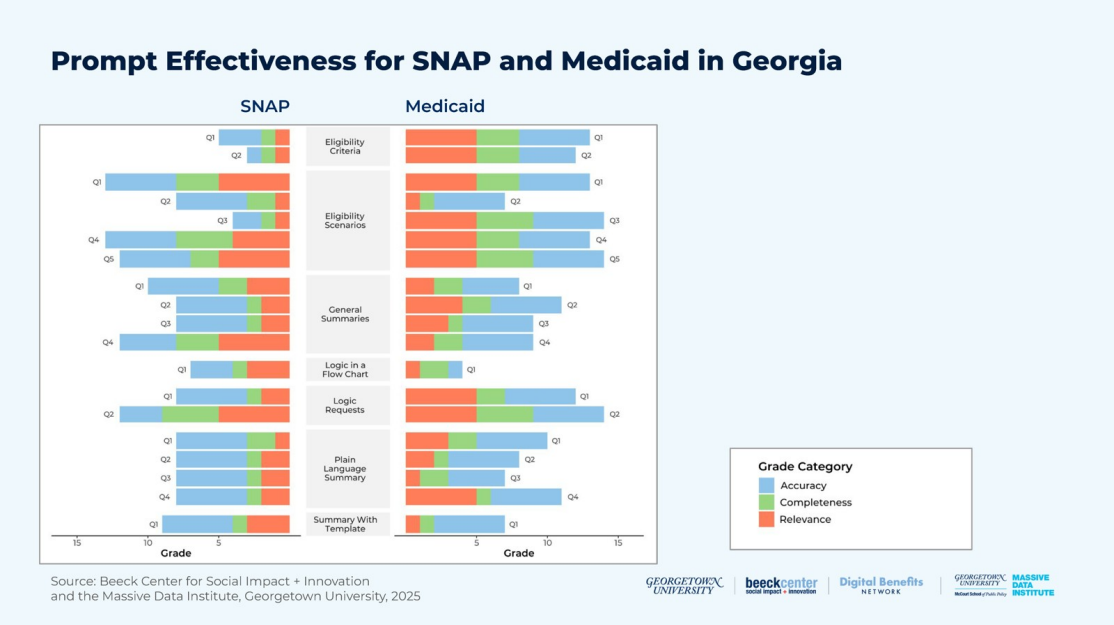
> 這些視覺化凸顯了 LLM 在不同問題類型上的表現，有些問題的得分高於其他問題。分數分為三個評估類別：準確性、完整性與相關性。需要注意的是，圖表中排除的問題類別得到「N/A」結果，表示該問題類別中的問題沒有可適用的答案。

Georgia

喬治亞州

Figure 10: Prompt Effectiveness for SNAP and Medicaid in Georgia

圖 10：喬治亞州 SNAP 與 Medicaid 的提示成效



For Georgia SNAP, the greatest variation was in the “What would determine if I am eligible” category. The lowest score came from a response to the questions, “Who is eligible?/What are the eligibility criteria?”, which received an overall score of 3. This means it scored 1 in all three categories: Accuracy (A), Completeness (C), and Relevance (R).

> 對喬治亞州 SNAP 而言，差異最大的是「什麼會決定我是否符合資格」這一類。最低分來自對「誰有資格？／資格標準是什麼？」這個問題的一則回應，總分為 3 分。這表示它在準確性（A）、完整性（C）與相關性（R）三個類別都只得 1 分。

Table 20: Example Results of Lowest Score from Georgia SNAP

表 20：喬治亞州 SNAP 最低分範例結果（表格內容過長，本譯本未收錄，請見原文）

It’s worth noting that none of the responses reached the maximum possible score of 15. However, two responses scored the highest for Georgia SNAP, with a total of 13 points.

> 值得注意的是，沒有任何回應達到滿分 15 分。不過，有兩則回應在喬治亞州 SNAP 中得到最高分，共 13 分。

Table 21: Example Results from Georgia Medicaid

表 21：喬治亞州 Medicaid 範例結果（表格內容過長，本譯本未收錄，請見原文）

For Georgia Medicaid, the lowest overall score across the categories was a 4. This score came from one response to the “Logic in a flow chart” prompt.

> 對喬治亞州 Medicaid 而言，各類別的最低總分是 4 分。這個分數來自對「流程圖中的邏輯」提示的一則回應。

Table 22: Example Results from Georgia Medicaid

表 22：喬治亞州 Medicaid 範例結果（表格內容過長，本譯本未收錄，請見原文）

Similarly, no responses achieved the maximum score of 15, but two responses scored 14, and four responses scored 13.

> 同樣地，沒有任何回應達到滿分 15 分，但有兩則回應得到 14 分，四則回應得到 13 分。

Pennsylvania

賓夕法尼亞州

Figure 11: Prompt Effectiveness for SNAP and Medicaid in Pennsylvania

圖 11：賓夕法尼亞州 SNAP 與 Medicaid 的提示成效

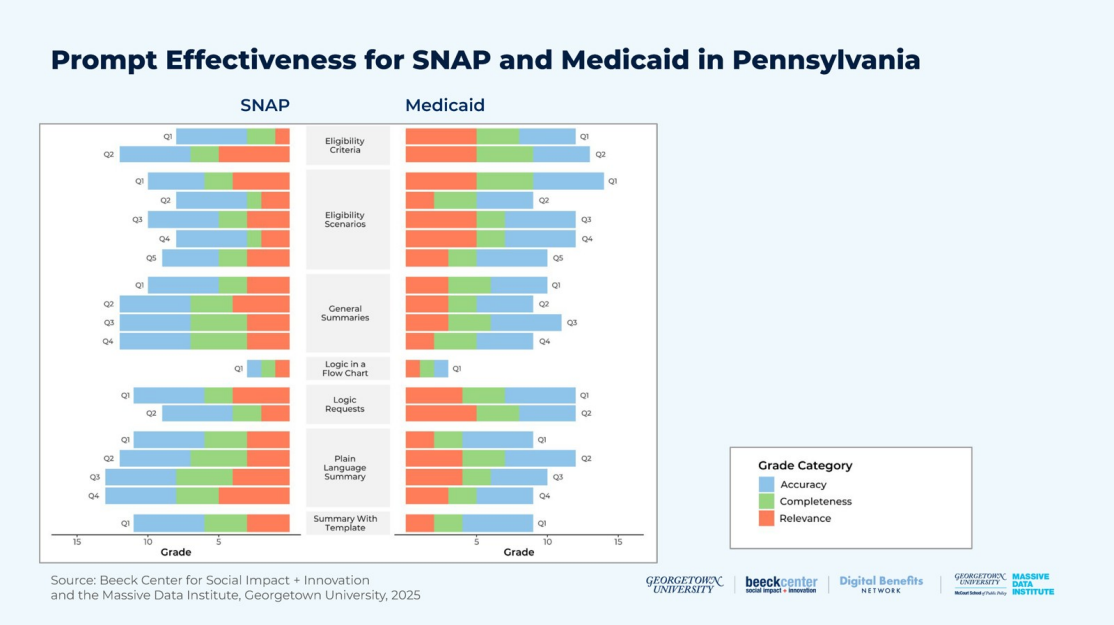


Table 23: Example Result from Pennsylvania SNAP

表 23：賓夕法尼亞州 SNAP 範例結果（表格內容過長，本譯本未收錄，請見原文）

For Pennsylvania SNAP, the greatest variation was observed in the “Summary” category. The lowest score was a 7, which came from a response to a “Summary” prompt.

> 對賓夕法尼亞州 SNAP 而言，差異最大的是「摘要」這一類。最低分是 7 分，來自對一則「摘要」提示的回應。

Table 24: Example Results from Pennsylvania SNAP

表 24：賓夕法尼亞州 SNAP 範例結果（表格內容過長，本譯本未收錄，請見原文）

None of the responses reached the maximum possible score of 15. However, two responses achieved 14 points, and five responses scored 13.

> 沒有任何回應達到滿分 15 分。不過，有兩則回應得到 14 分，五則回應得到 13 分。

Table 25: Example Result from Pennsylvania Medicaid

表 25：賓夕法尼亞州 Medicaid 範例結果（表格內容過長，本譯本未收錄，請見原文）

For Pennsylvania Medicaid, the lowest overall score across the three categories was a 9. This score came from one response to the “plain language summary” prompt.

> 對賓夕法尼亞州 Medicaid 而言，三個類別的最低總分是 9 分。這個分數來自對「白話摘要」提示的一則回應。

Table 26: Example Result from Pennsylvania Medicaid

表 26：賓夕法尼亞州 Medicaid 範例結果（表格內容過長，本譯本未收錄，請見原文）

Similarly, no responses reached the maximum score of 15, only one response achieved a 14.

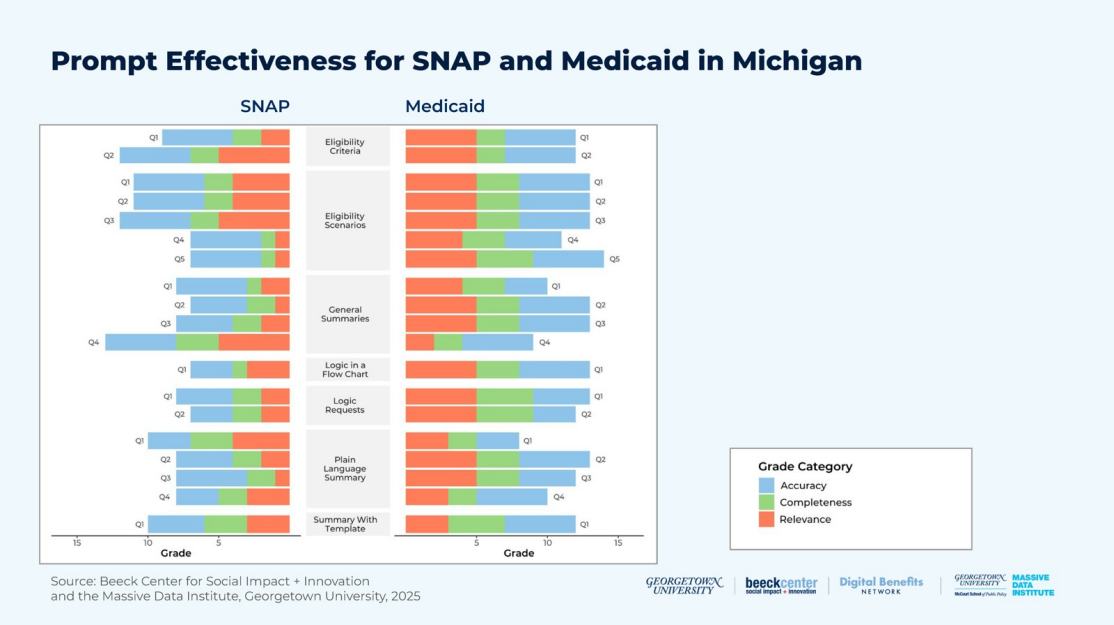
> 同樣地，沒有任何回應達到滿分 15 分，只有一則回應得到 14 分。

Michigan

密西根州

Figure 12: Prompt Effectiveness for SNAP and Medicaid in Michigan

圖 12：密西根州 SNAP 與 Medicaid 的提示成效



For Michigan SNAP, the greatest variation was observed in the “Summary” category. The lowest score was a 7, which came from one response to a “Summary” prompt, one to a “Logic” prompt, and one to the “What would determine if I am eligible?” prompt. None of the responses reached the maximum possible score of 15, and only a few responses scored as high as 13.

> 對密西根州 SNAP 而言，差異最大的是「摘要」這一類。最低分是 7 分，來自一則對「摘要」提示、一則對「邏輯」提示，以及一則對「什麼會決定我是否符合資格？」提示的回應。沒有任何回應達到滿分 15 分，只有少數回應高達 13 分。

For Michigan Medicaid, the greatest variation was observed in the “Plain language summary” category. The lowest overall score across the three categories was an 8, which came from one response to a “Plain language summary” prompt. Similarly, no responses reached the maximum score of 15, but seven responses achieved a score of 13.

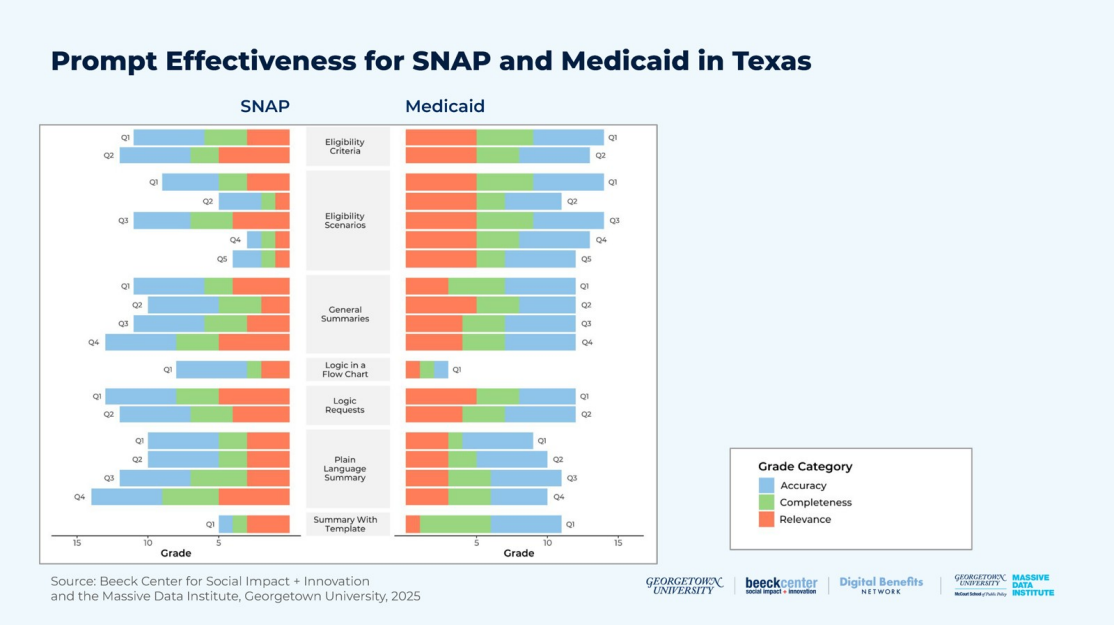
> 對密西根州 Medicaid 而言，差異最大的是「白話摘要」這一類。三個類別的最低總分是 8 分，來自對一則「白話摘要」提示的回應。同樣地，沒有任何回應達到滿分 15 分，但有七則回應得到 13 分。

Texas

德州

Figure 13: Prompt Effectiveness for SNAP and Medicaid in Texas

圖 13：德州 SNAP 與 Medicaid 的提示成效



For Texas SNAP, the greatest variation was observed in the “Summary” category. The lowest score was a 4, which came from one response to the “What would determine if I am eligible?” prompt. None of the responses achieved the maximum score of 15, but one response scored 14, and two responses scored 13.

> 對德州 SNAP 而言，差異最大的是「摘要」這一類。最低分是 4 分，來自對「什麼會決定我是否符合資格？」提示的一則回應。沒有任何回應達到滿分 15 分，但有一則回應得到 14 分，兩則回應得到 13 分。

For Texas Medicaid, the greatest variation was found in the “What would determine if I am eligible?” category. The lowest overall score across the three categories was a 9, which came from one response to a “Plain language summary” prompt. Similarly, no responses reached the maximum score of 15, but three responses scored 14, and two scored 13.

> 對德州 Medicaid 而言，差異最大的是「什麼會決定我是否符合資格？」這一類。三個類別的最低總分是 9 分，來自對一則「白話摘要」提示的回應。同樣地，沒有任何回應達到滿分 15 分，但有三則回應得到 14 分，兩則得到 13 分。

Considerations for Public Benefits Use Cases

對公共福利使用情境的考量

While we applied these methods to specific states and policies, there are considerations for wider public benefits use cases.

> 雖然我們把這些方法套用在特定的州與政策上，但對更廣泛的公共福利使用情境也有一些考量。

State governments can make it easier for their policies to be used in LLMs by making them digitally-accessible. At a minimum, this is a single PDF that allows text to be extracted; even better is a plain text or HTML document or

webpage that presents the policy in full. One excellent example of digitally-accessible policies is Oklahoma. The Oklahoma Rules website allows users to view and download the HTML of the entire administrative code for the state.

> 州政府可以把政策做成數位化、易於取用的形式，讓政策更容易被 LLM 使用。最基本的做法是提供一份可擷取文字的單一 PDF；更好的做法是提供完整呈現政策的純文字或 HTML 文件或網頁。數位化可取用政策的一個絕佳範例是奧克拉荷馬州。奧克拉荷馬州規則網站讓使用者可以檢視並下載該州整部行政法規的 HTML。

Focusing LLMs on specific policy documents increases accuracy in responses, but results are mixed for relevance and completeness. Simply pointing an LLM to an authoritative document does not mean it will pull out the relevant information from the document. A more promising direction may be to consider one of the following: custom-designed RAG document collections, fine-tuning with labeled data, or newer reinforcement learning approaches that automatically attempt to identify the relevant information for the query. Additionally, the way policy documents are written and structured—such as including specific eligibility criteria tables—can also improve LLM performance.

> 讓 LLM 聚焦於特定的政策文件，可以提高回應的準確性，但在相關性與完整性上結果不一。單純把 LLM 指向一份權威文件，並不代表它會從文件中抽出相關資訊。比較有希望的方向，可能是考慮下列其中一種做法：自訂設計的 RAG 文件集合、以標註資料進行微調，或是較新的強化學習方法，自動嘗試找出與查詢相關的資訊。此外，政策文件的撰寫與結構方式（例如納入具體的資格標準表格），也可以提升 LLM 的表現。

Experiment 3: Using LLMs to Generate Machine-Readable Rules

實驗三：運用 LLM 產生機器可讀規則

Motivation and Question

動機與問題

In this experiment, we examined the capability of LLMs to automate an essential component of Rules as Code solutions. Specifically, we assessed the ability of LLMs to systematically extract rules from official policy manuals and encode them in a machine-readable format. This type of experiment supported our goal of enabling efficient implementation of digitized policy. If rules could be successfully generated, then a rules engine could be developed more efficiently and programmatically integrated into broader software applications. Due to the inherently unstructured nature of policy documents, we recognized the need for a structured approach to access rules in a standardized format. This type of standardization would create consistency and applicability across different states and programs, while facilitating a uniform evaluation process for the generated rules in terms of accuracy and completeness.

> 在這項實驗中，我們檢視 LLM 自動化「規則即程式碼」解決方案中一個關鍵環節的能力。具體而言，我們評估 LLM 系統性地從官方政策手冊擷取規則，並以機器可讀格式編碼的能力。這類實驗支持我們的目標：讓政策的數位化能有效率地實施。如果能成功產生規則，就可以更有效率地開發規則引擎，並以程式化方式整合進更廣泛的軟體應用。由於政策文件本質上是非結構化的，我們認識到需要一種結構化的方法，以標準化格式取得規則。這種標準化可以讓不同的州與計畫之間具備一致性與適用性，同時便於對所生成規則的準確性與完整性進行統一的評估。

The main guiding questions of the study are:

> 本研究的主要引導問題為：

- Do LLMs reliably extract rules from unstructured policy documents when guided by a structured template?

> - 在結構化範本的引導下，LLM 能否可靠地從非結構化的政策文件中擷取規則？

- How does the use of a structured rules template impact an LLM's ability to produce relevant and accurate output?

> - 使用結構化規則範本，如何影響 LLM 產出相關且準確結果的能力？

- What performance differences emerge in extracting policy rules when using a structured template with a plain-prompting approach versus a RAG framework?

> - 在擷取政策規則時，「結構化範本搭配一般提示」與「RAG 框架」之間，會出現什麼表現差異？

Rules Templating

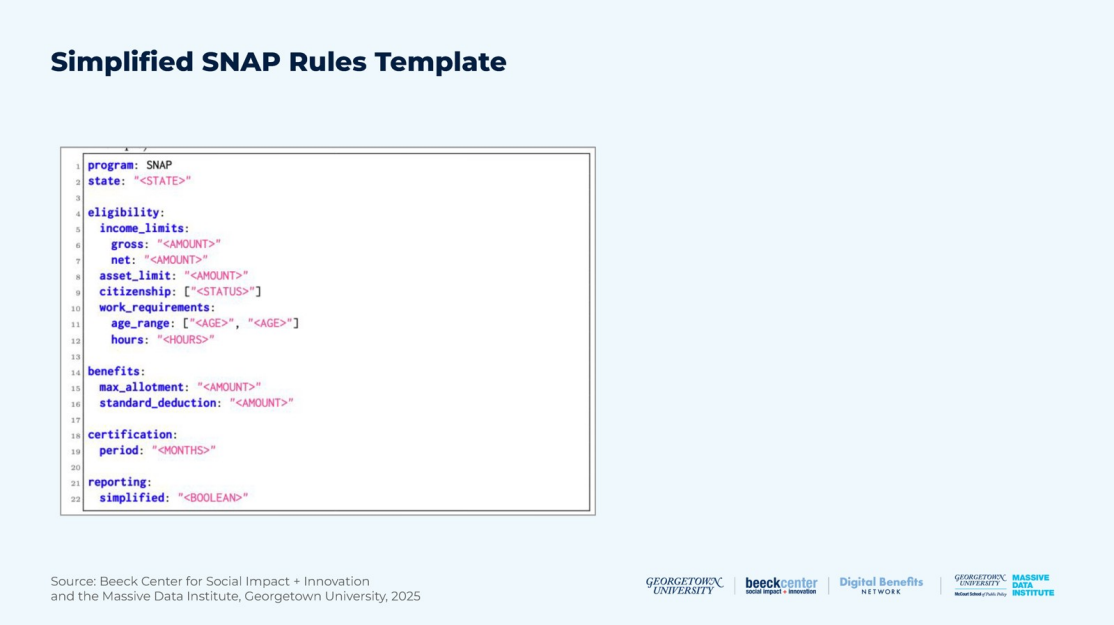
規則範本化

Initial attempts to prompt the LLM to directly identify rules from policy documents proved unreliable, as outputs were highly sensitive to prompt phrasing. We came to the conclusion that the LLM struggled to produce dependable and uniform results without a strictly-defined rules template. We conducted the remaining experiments with a manually-curated rules template for both SNAP and Medicaid across six states. For SNAP, the rules included criteria such as income limits based on family size, citizenship requirements, and other key eligibility conditions. For Medicaid, the rules incorporated covered groups (e.g., pregnant women, low-income families) and age requirements, among other criteria. Figure 14 presents an example of a simplified rules template used for SNAP.

> 一開始嘗試直接提示 LLM 從政策文件中找出規則，結果並不可靠，因為輸出對提示措辭高度敏感。我們得出的結論是：沒有嚴格定義的規則範本，LLM 難以產出可靠且一致的結果。我們其餘的實驗，針對六個州的 SNAP 與 Medicaid，都使用人工整理的規則範本進行。對 SNAP 而言，規則包括依家戶人數而定的收入上限、公民身分要求，以及其他關鍵的資格條件。對 Medicaid 而言，規則包括涵蓋族群（例如孕婦、低收入家庭）、年齡要求，以及其他標準。圖 14 呈現 SNAP 所用簡化規則範本的範例。

Figure 14: Simplified SNAP Rules Template

圖 14：簡化的 SNAP 規則範本

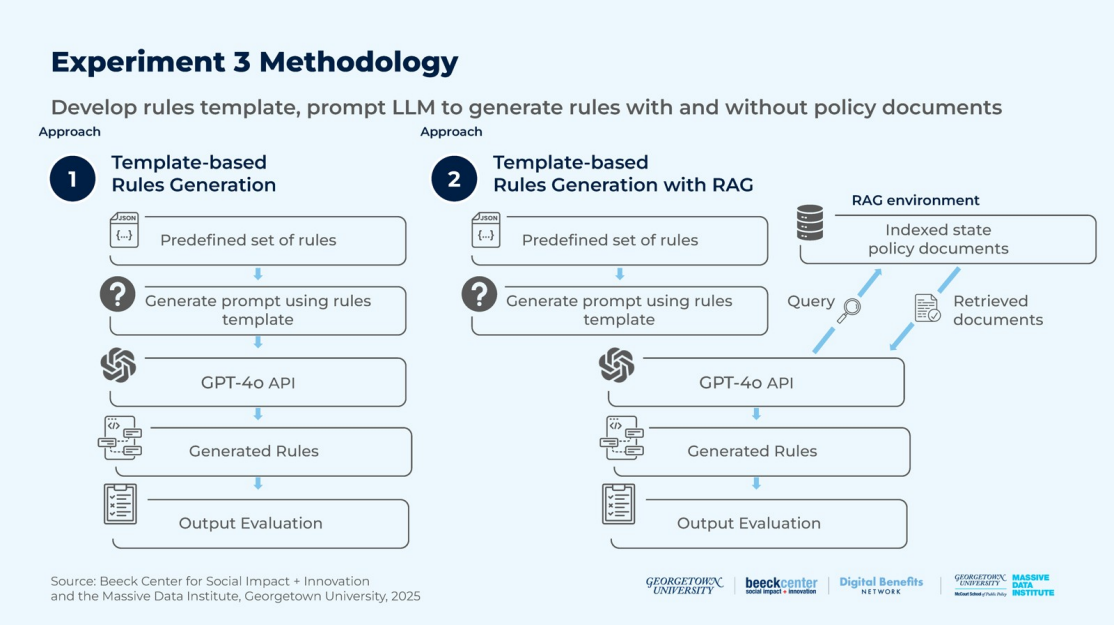


Methodology

方法

Figure 15: Experiment 3 Methodology

圖 15：實驗三方法



Our task in this experiment involved prompting the LLM to populate the predefined rules with corresponding values based on a specified program and state. The goal was to compare the ability of readily available LLMs and LLMs with additional external knowledge to generate programmatic policy rules. To this end, we evaluated two distinct approaches using GPT-4o as the LLM.

> 我們在這項實驗中的任務，是提示 LLM 依指定的計畫與州別，把預先定義的規則填入對應的數值。目標是比較「現成的 LLM」與「具備額外外部知識的 LLM」生成程式化政策規則的能力。為此，我們以 GPT-4o 作為 LLM，評估兩種不同的做法。

We considered a direct query approach that was unsuccessful and led to an experimental design that began by compiling a rules template, where we assembled a predefined set of policy rules in a structured JSON format. These rules were selected based on their common applicability across different programs and state-specific variations in our case studies. Below, we describe the workflow of both approaches, illustrated in Figure 15.

> 我們曾考慮直接查詢的做法，但並不成功，因而促成了一個實驗設計：先編製一份規則範本，以結構化的 JSON 格式整理出預先定義的一組政策規則。這些規則是依據它們在不同計畫間的共通適用性，以及我們個案研究中各州的差異來挑選。下文說明兩種做法的工作流程，如圖 15 所示。

A. Template-Based Rules Generation (Plain Prompting)

A. 以範本為基礎的規則生成（一般提示）

Prompt Construction: We formulated a prompt for generating rules using the rules template and task-specific instructions.

> 建構提示：我們運用規則範本與任務特定指示，撰寫用於生成規則的提示。

Rules Generation: We provided the curated prompts to the LLM (GPT-4o), which generated values for the policy rules relying on its own training data.

> 規則生成：我們把整理好的提示交給 LLM（GPT-4o），它依靠自身的訓練資料生成政策規則的數值。

Output Evaluation: We evaluated the produced rules against the original values manually-extracted from authoritative sources (e.g., state policy manuals).

> 輸出評估：我們把產出的規則，與從權威來源（例如州政策手冊）人工擷取的原始數值相比對評估。

B. Template-Based Rules Generation with RAG

B. 以範本為基礎、搭配 RAG 的規則生成

Document Indexing: We collected official policy documents and organized them in a searchable format. The process involved breaking the documents into fixed-size chunks, which were then converted into vector embeddings, or a numerical representation of the data that could be used by deep learning models. By mapping vectors into common embedding spaces, we then assessed how similar two documents (text segments) were, based on their semantic meaning. The goal of this step was to access the relevant information from the policy document collection to improve the final response provided by the LLM.

> 文件索引：我們蒐集官方政策文件，並整理成可搜尋的格式。過程包括把文件切成固定大小的區塊，再轉成向量嵌入（vector embeddings），也就是可供深度學習模型使用的資料數值表示。把向量對應到共同的嵌入空間後，我們便能依語意評估兩份文件（文字片段）的相似程度。這個步驟的目的，是從政策文件集合中取得相關資訊，以改善 LLM 最終提供的回應。

Prompt Construction: The rules generation prompt was formulated using the rules template and task-specific instructions. This prompt was used twice:

> 建構提示：規則生成提示是運用規則範本與任務特定指示所撰寫。這個提示被使用了兩次：

- First, the LLM was instructed to generate search queries to retrieve relevant documents from the RAG database.

> - 第一次，指示 LLM 產生搜尋查詢，以便從 RAG 資料庫中檢索相關文件。

- Once the relevant documents were retrieved, they were appended to the generation prompt, and the LLM was instructed to generate the policy rules.

> - 檢索到相關文件後，把它們附加到生成提示中，並指示 LLM 生成政策規則。

RAG Queries and Document Retrieval: The LLM-generated search queries were then used to find the most semantically-relevant text segments in the policy database. The retrieved segments were appended to the rules generation prompt.

> RAG 查詢與文件檢索：接著使用 LLM 產生的搜尋查詢，在政策資料庫中找出語意上最相關的文字片段。檢索到的片段被附加到規則生成提示中。

Rules Generation: After incorporating the relevant policy information into the prompt, the LLM generated policy rules guided by the retrieved policy excerpts.

> 規則生成：把相關政策資訊納入提示後，LLM 在所檢索政策摘錄的引導下生成政策規則。

Output Evaluation: We evaluated the produced rules against manually-extracted rules values and verified the output from authoritative sources.

> 輸出評估：我們把產出的規則，與人工擷取的規則數值相比對，並依權威來源驗證輸出。

Developments and Challenges

發展與挑戰

Template Design: Developing a comprehensive rules template to capture the evolving, state-specific nuances of public benefits policies was challenging. However, manually developing such a template remained a critical first step to standardize outputs and enable systematic rule extraction.

> 範本設計：要開發一份完整的規則範本，來涵蓋公共福利政策隨時演變、各州不同的細微差異，並不容易。然而，人工開發這樣的範本，仍是標準化輸出並實現系統性規則擷取的關鍵第一步。

Chunking and Context Overhead: Another big challenge with the RAG framework involved compiling a manageable yet comprehensive set of resources for retrieval. As noted in Experiment 2, state-issued policy manuals, which sometimes spanned thousands of pages, presented a retrieval challenge due to their sheer volume and complexity. To mitigate this, large chunks of text were regularly retrieved to maximize the chances of including only relevant information in the LLM's input. However, this strategy created a trade-off: While longer context increased the likelihood of capturing relevant details, they also degraded the LLM's performance by overwhelming its processing capacity—a challenge commonly referred to as the "needle in the haystack." In other words, the model struggled to find critical information buried in a large volume of less relevant data. In contrast, retrieving fewer documents reduced context size but increased the risk of missing key information.

> 切分區塊與情境負擔：RAG 框架的另一項重大挑戰，是編製一組大小可處理、卻又夠完整的檢索資源。如實驗二所述，各州發布的政策手冊有時長達數千頁，其龐大的篇幅與複雜度造成檢索上的挑戰。為了緩解這個問題，我們經常檢索較大的文字區塊，以提高讓 LLM 的輸入只包含相關資訊的機會。然而，這個策略有所取捨：較長的情境雖然提高了擷取到相關細節的可能性，卻也因為壓垮 LLM 的處理能力而降低其表現，這個挑戰通常被稱為「大海撈針」（needle in the haystack）。換言之，模型很難在大量較不相關的資料中找到埋藏其中的關鍵資訊。相反地，檢索較少的文件雖然縮小了情境，卻增加遺漏關鍵資訊的風險。

Information Scattering and Retrieval: Policy details relevant to a single rule were sometimes scattered across multiple sections of a manual. In such cases, a document re-ranking strategy—which improved retrieval accuracy by re-scoring the initially-retrieved documents using advanced semantic similarity—became essential to prioritize the most relevant segments for retrieval. While effective, this approach introduced additional computational overhead and complexity in generation.

> 資訊分散與檢索：與單一規則相關的政策細節，有時會分散在手冊的多個章節。在這種情況下，文件重新排序策略（藉由進階語意相似度，對最初檢索到的文件重新評分，以提高檢索準確度）就變得不可或缺，用來優先處理最相關的片段。這種做法雖然有效，卻也增加了額外的運算負擔與生成上的複雜度。

Findings

發現

For Georgia, Pennsylvania, and Texas, we generated rules that represented policies associated with state-specific SNAP programs. These rules were grouped into higher-level categories (e.g., income, asset limit). For each category, we calculated the average accuracy of the generated rules for each state and approach. The results are presented in Figure 16.

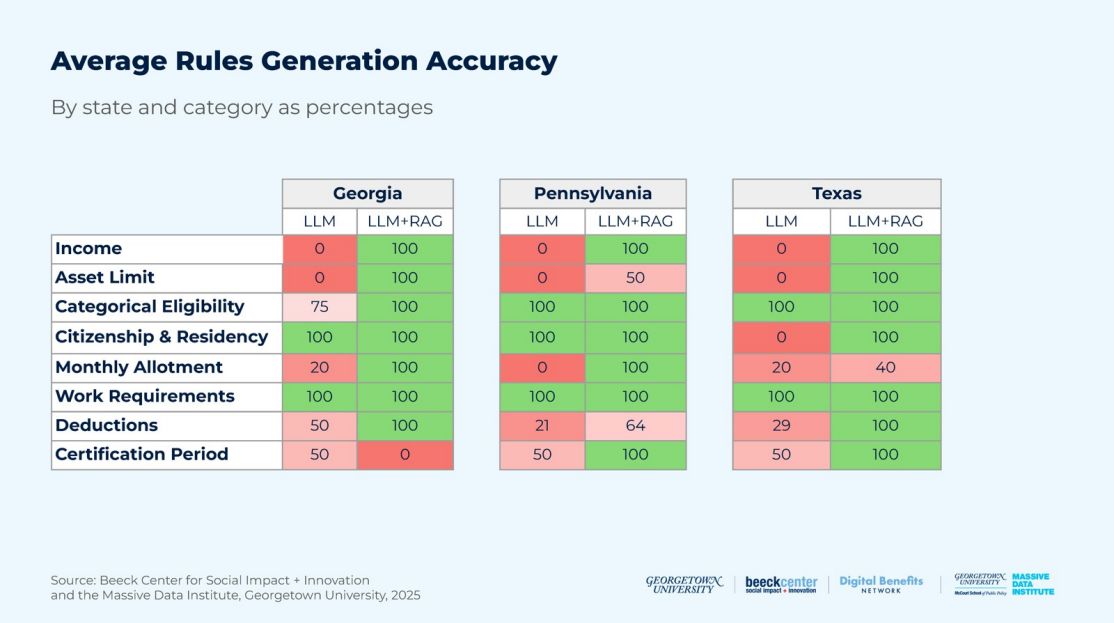
> 針對喬治亞州、賓夕法尼亞州與德州，我們生成了代表各州特定 SNAP 計畫相關政策的規則。這些規則被歸入較高層次的類別（例如收入、資產上限）。針對每個類別，我們計算了各州與各做法下所生成規則的平均準確度。結果呈現於圖 16。

A detailed spreadsheet of the results are included in the Appendix.

> 詳細的結果試算表收錄於附錄。

Figure 16: Average Rules Generation Accuracy by State and Category

圖 16：依州別與類別的平均規則生成準確度



	Georgia		Pennsylvania		Texas	
	LLM	LLM+RAG	LLM	LLM+RAG	LLM	LLM+RAG
Income	0	100	0	100	0	100
Asset Limit	0	100	0	50	0	100
Categorical Eligibility	75	100	100	100	100	100
Citizenship & Residency	100	100	100	100	0	100
Monthly Allotment	20	100	0	100	20	40
Work Requirements	100	100	100	100	100	100
Deductions	50	100	21	64	29	100
Certification Period	50	0	50	100	50	100

Notable results in this table include:

> 此表中值得注意的結果包括：

Plain prompting yielded unreliable generation accuracy for state-specific rules. The plain prompting approach without the additional external knowlegde(without RAG) demonstrated significant limitations with numerical rules like income and asset limits, achieving 0% accuracy in these categories across all states. Conversely, categories that were more likely to be consistent across states, such as citizenship and work requirements, were generated with 100% accuracy (except for citizenship in Texas, where the plain LLM approach failed to provide the correct residency requirement).

> 一般提示在各州特定規則的生成準確度上並不可靠。沒有外部知識（不使用 RAG）的一般提示做法，在收入與資產上限這類數值型規則上顯現明顯的限制，在所有州的這些類別中準確度皆為 0%。相反地，較可能在各州之間一致的類別，如公民身分與工作要求，則以 100% 的準確度生成（德州的公民身分除外，一般 LLM 做法在該處未能給出正確的居住要求）。

Other categories, such as standard deductions and certification periods, yielded mixed success. Challenges were primarily due to the LLM's reliance on training data which lacked the granularity required for producing localized state variations. However, for rules that remained uniform across states, such as federally-mandated rules (citizenship and categorical eligibility) the LLM was able to retrieve information accurately from its native knowledge base.

> 其他類別，如標準扣除額與認證期間，成功程度不一。挑戰主要來自 LLM 依賴的訓練資料缺乏產出在地化州別差異所需的細緻度。不過，對於在各州之間維持一致的規則，例如聯邦強制規定的規則（公民身分與類別資格），LLM 能夠從其原有的知識庫中準確取得資訊。

RAG enhanced the alignment of generated rules with policy documents. The RAG approach effectively addressed the shortcomings of the plain prompting method by providing the additional context necessary to enhance both generation accuracy and timeliness. With this framework, the LLM successfully generated all predefined rules with 100% accuracy, except for a few cases such as the certification period in Georgia, deductions in Pennsylvania, and the monthly allotment in Texas. These results, as shown in Figure 16, highlight the clear difference in performance between the two approaches.

> RAG 提升了所生成規則與政策文件的吻合度。RAG 做法有效地補足了一般提示方法的缺點，藉由提供額外的情境，同時提升生成的準確度與時效性。在這個框架下，LLM 成功以 100% 的準確度生成所有預先定義的規則，只有少數例外，例如喬治亞州的認證期間、賓夕法尼亞州的扣除額，以及德州的每月給付額。如圖 16 所示，這些結果凸顯出兩種做法在表現上的明顯差異。

Missing context in RAG led to erroneous outputs. A closer analysis of the certification period category in Georgia revealed that the inaccurate generation using the RAG framework was a consequence of the absence of relevant information from our policy database—excluding key context from the LLM's input. This highlighted a unique challenge introduced by the RAG framework: In general, the LLM relies heavily on provided context and less on pretrained knowledge. As a result, when required information was missing from the context, the model was more prone to generating erroneous outputs, as it assumed the answer existed in the retrieved text.

> RAG 中缺少情境會導致錯誤輸出。深入分析喬治亞州「認證期間」這個類別後發現，使用 RAG 框架所產生的不準確結果，是因為我們的政策資料庫缺少相關資訊，使 LLM 的輸入少了關鍵情境。這凸顯 RAG 框架帶來的一項獨特挑戰：一般而言，LLM 高度依賴所提供的情境，較少仰賴預訓練的知識。因此，當情境中缺少所需資訊時，模型更容易產生錯誤輸出，因為它假定答案就存在於所檢索到的文字中。

Considerations for Public Benefits Use Cases

對公共福利使用情境的考量

Structured templates are essential for rules extraction. The success of machine-readable rules generation is, to a large degree, dependent on the use of standardized templates. Public benefits agencies should collaborate to develop shared schemas that capture multiple levels of eligibility criteria. The absence of such templates results in significantly higher levels of inaccuracy and inconsistency in LLM output, further complicating interoperability with other components of an automated system.

> 規則擷取需要結構化範本。機器可讀規則的生成能否成功，在很大程度上取決於是否使用標準化範本。公共福利機關應合作制定共用的結構（schema），以涵蓋多層次的資格標準。缺少這類範本，會使 LLM 輸出明顯更不準確、更不一致，並進一步使其與自動化系統其他元件的互通更加複雜。

Rules generation can provide a structure for code generation. Given the success of rules generation for most contexts in this experiment, separating the rule generation from the code generation is a promising direction. While there is a need to begin with a manually-created template, once that is created, it can be part of a template library that can be used to generate the rules. The generated rules act as a structured skeleton of the code, providing the necessary logic for cleaner, more robust code generation. This multi-step, LLM-assisted implementation is a promising direction because policies in different states are inconsistently written. This variation means that translating certain highly variable policies directly to code may not be as promising a path for complicated policies as generating rules.

> 規則生成可為程式碼生成提供結構。鑑於本實驗中規則生成在大多數情境下的成功，把規則生成與程式碼生成分開，是一個有希望的方向。雖然需要從人工建立的範本開始，但一旦建立，它就可以成為範本庫的一部分，用來生成規則。所生成的規則就像程式碼的結構化骨架，為更乾淨、更穩健的程式碼生成提供必要的邏輯。這種多步驟、由 LLM 輔助的實作是個有希望的方向，因為各州的政策寫法並不一致。這種差異意味著，對於複雜的政策，把某些高度多變的政策直接轉成程式碼，可能不如先生成規則來得有希望。

Regular policy changes impact LLM output. To keep up with the recurring policy and legislative changes of public benefits programs, automated Rules as Code systems must remain up to date. This involves closely monitoring the LLM's output to ensure it reflects the latest policy amendments. This might require scheduled model fine-

tuning and updating the policy library used for document retrieval. Requiring the LLM to provide a reference for the generated rules may help with this as well.

> 定期的政策變動會影響 LLM 的輸出。為了跟上公共福利計畫反覆出現的政策與立法變動，自動化的「規則即程式碼」系統必須保持更新。這需要密切監控 LLM 的輸出，確保它反映最新的政策修正。這可能需要排定模型的微調，並更新用於文件檢索的政策庫。要求 LLM 為所生成的規則提供出處，或許也有幫助。

LLMs must handle dependencies across different benefits eligibility criteria. Benefits programs frequently involve interconnected requirements. For instance, a household's gross income limits may vary depending on its composition (e.g., whether a member is elderly or disabled). If the LLM failed to account for such variations, it would lead to incorrect eligibility determination in automated systems. Therefore, careful handling of these dependencies is essential to prevent inaccuracies in LLM-powered automations. This could be addressed through detailed audits using test cases that feature intricate policy rules. A complementary approach would be to consider incorporating knowledge graphs into the RAG input, either independently or in combination with a traditional vector database.

> LLM 必須處理不同福利資格標準之間的相依關係。福利計畫經常涉及彼此關聯的要求。例如，家戶的總收入上限可能依其組成而異（例如是否有成員為長者或身心障礙者）。如果 LLM 沒有考慮到這類差異，就會在自動化系統中導致錯誤的資格判定。因此，謹慎處理這些相依關係，對於防止 LLM 驅動的自動化出現不準確至關重要。可以藉由使用含有複雜政策規則的測試案例進行詳細稽核來因應。一個互補的做法，是考慮將知識圖譜（knowledge graph）納入 RAG 的輸入，可單獨使用，也可與傳統向量資料庫合併使用。

Policy documents need to be accessible by machines. Policy accessibility directly impacts accuracy in LLM-powered systems. In addition to making policies more accessible for humans, state governments should advocate for digitizing policies in structured formats and adopting more modular and easily searchable documents to improve their ability to be read by machines. This would allow these systems to easily localize critical information in lengthy manuals and reduce retrieval complexity.

> 政策文件需要能被機器讀取。政策的可取得性直接影響 LLM 驅動系統的準確度。除了讓人類更容易取得政策之外，州政府還應倡導把政策數位化為結構化格式，並採用更模組化、更容易搜尋的文件，以提升機器讀取它們的能力。這能讓這些系統輕易在冗長的手冊中定位關鍵資訊，並降低檢索的複雜度。

Experiment 4: Using LLMs to Generate Software Code

實驗四：運用 LLM 產生軟體程式碼

Motivation and Question

動機與問題

In this experiment, we aimed to evaluate the ability of LLMs to generate software code, specifically for determining eligibility status in benefits programs, by building on insights from earlier phases of this study. Our primary objective was to understand the feasibility of using LLMs to automate code generation for key components of the benefits delivery process. By doing this, we began exploring how such automation could simplify access to benefits for eligible individuals, assist organizations in optimizing service workflows, and contribute to the development of more efficient and adaptable systems for managing benefits programs.

> 在這項實驗中，我們以本研究前幾個階段的洞見為基礎，旨在評估 LLM 生成軟體程式碼的能力，特別是用於判定福利計畫中的資格狀態。我們的主要目標是了解，用 LLM 自動化福利提供流程中關鍵環節的程式碼生成，是否可行。藉此，我們開始探索這種自動化如何能簡化符合資格者取得福利的過程、協助組織優化服務工作流程，並促進開發更有效率、更具彈性的福利計畫管理系統。

The main questions guiding our study were:

> 引導本研究的主要問題為：

- How did different prompting strategies impact the accuracy and usability of LLM-generated code for eligibility determination?

> - 不同的提示策略如何影響 LLM 為資格判定所生成程式碼的準確性與可用性？

- Did LLMs synthesize end-to-end logic from unstructured policy documents into functional software code, and what were the key limitations in modeling interdependencies between eligibility criteria?

> - LLM 是否能把非結構化政策文件中的端到端邏輯，綜合成可運作的軟體程式碼？在建模資格標準之間的相依關係上，主要的限制是什麼？

- Did an iterative, multi-step approach reduce errors compared to direct code generation, or did it introduce compounding inaccuracies?

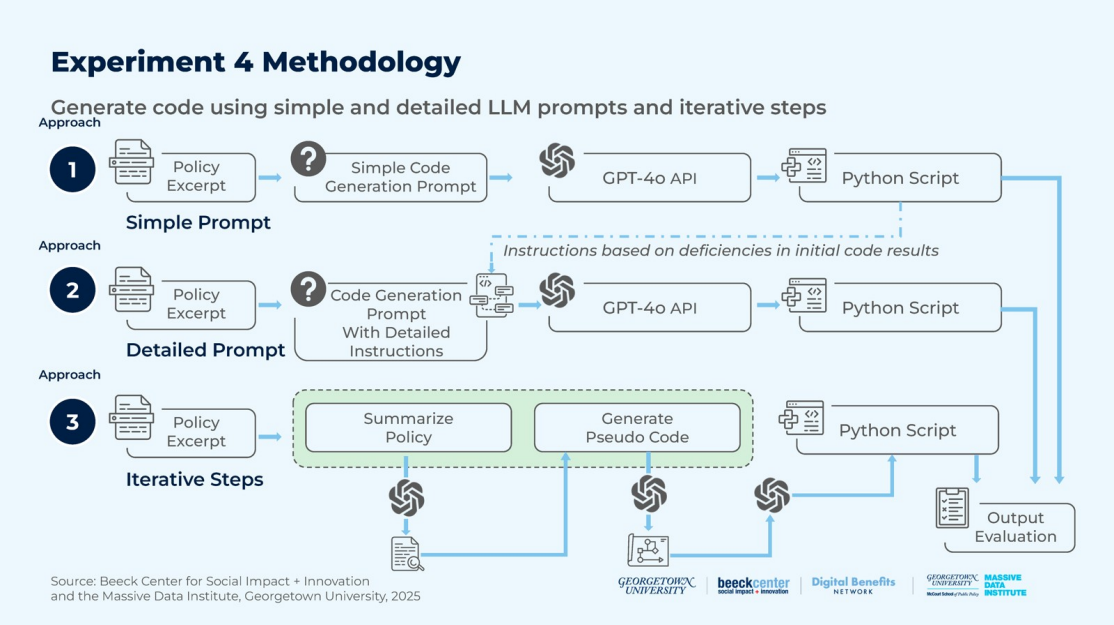
> - 相較於直接生成程式碼，反覆迭代的多步驟做法是否減少了錯誤，還是引入了累積性的不準確？

Methodology

方法

Figure 17: Experiment 4 Methodology

圖 17：實驗四方法



We developed three distinct experimental designs to assess how different inputs and prompting strategies impacted the quality and effectiveness of LLM-generated software code. Our goal was to determine the optimal input structure and level of guidance required by LLMs to generate accurate and functional code.

> 我們發展了三種不同的實驗設計，用來評估不同的輸入與提示策略如何影響 LLM 所生成軟體程式碼的品質與成效。我們的目標是判定，LLM 要生成準確且可運作的程式碼，最理想的輸入結構與所需的引導程度。

To control for potential confounding factors, we designed a small-scale prototype task that focused on a single program and state. Specifically, we prompted the LLM to generate software code to determine an applicant's SNAP worker registration status in Georgia. This targeted experiment allowed us to closely examine, in a controlled setting, the LLM's code-generation capabilities. For this experiment, we used GPT-4o to evaluate the different prompting strategies. We tested the following three prompting strategies in this task. Figure 17 illustrates the workflow of each experimental design.

> 為了控制可能的干擾因素，我們設計了一個小規模的原型任務，聚焦於單一計畫與單一州別。具體而言，我們提示 LLM 生成軟體程式碼，用來判定喬治亞州申請人的 SNAP 工作登記狀態。這項針對性的實驗，讓我們得以在受控的環境中仔細檢視 LLM 的程式碼生成能力。在這項實驗中，我們使用 GPT-4o 來評估不同的提示策略。我們在此任務中測試了下列三種提示策略。圖 17 說明了每種實驗設計的工作流程。

A. Design 1: Simple Prompt (Baseline)

A. 設計一：簡單提示（基準）

Expert knowledge construction: In order to provide state-specific policy knowledge, we added a policy excerpt as input to the LLM. This excerpt was extracted from Georgia's official SNAP manual and was manually-verified to include all necessary information for determining an applicant's worker registration status, given their attributes (e.g., age and employment status). This served as the primary reference for the LLM to infer policy rules and code logic.

> 專家知識建構：為了提供各州特定的政策知識，我們把一段政策摘錄作為 LLM 的輸入。這段摘錄取自喬治亞州官方 SNAP 手冊，並經人工驗證包含依申請人屬性（例如年齡與就業狀態）判定其工作登記狀態所需的全部資訊。它成為 LLM 推論政策規則與程式邏輯的主要參考。

Minimalistic design prompt construction: For this experimental design, we used minimal instructions in the prompt to guide the LLM's code generation. This approach allowed us to assess the LLM's capability to autonomously interpret the policy text and extract the relevant, underlying logic without additional guidance.

> 極簡設計的提示建構：在這個實驗設計中，我們在提示中只用最少的指示來引導 LLM 生成程式碼。這種做法讓我們能評估 LLM 在沒有額外引導下，自行詮釋政策文字並擷取相關底層邏輯的能力。

B. Design 2: Detailed Prompt (Guided Approach)

B. 設計二：詳細提示（引導式做法）

Expert knowledge construction: We provided the policy excerpt, along with a more structured prompting template featuring detailed instructions.

> 專家知識建構：我們提供政策摘錄，以及一份具備詳細指示、結構更完整的提示範本。

Expanded design prompt construction: We expanded the instructions to address the observed shortcomings from the baseline, e.g., incomplete decision-making logic, inadequate input handling, or a lack of modularity in the generated code. The detailed guidance emphasized improving code structure and enhancing the LLM's ability to correctly interpret and apply the rules. Table 27 shows the prompt template used for the guided approach.

> 擴充的設計提示建構：我們擴充了指示，以處理在基準做法中觀察到的缺點，例如決策邏輯不完整、輸入處理不足，或生成的程式碼缺乏模組化。詳細的引導著重於改善程式碼結構，並提升 LLM 正確詮釋與套用規則的能力。表 27 顯示引導式做法所用的提示範本。

Table 27: Example Guided Prompt

表 27：引導式提示範例（表格內容保留英文原文）

System Prompt	User Prompt
You are an expert policy analyst and a skilled software developer expert working on a project to digitize the US benefits policies by translating complex policy text into software code to automate the benefits eligibility determination process.	You are tasked with writing a Python script to determine the work registration status of an applicant for SNAP benefits in {STATE}. You are provided with official policy documentation that covers the rules and regulations regarding the work registration requirements. Follow

	these steps carefully: 1. Extract Relevant Inputs: – Thoroughly analyze the policy documentation. – Identify and list all inputs required to determine the work registration status. 2. Model Logical Relationships: – Define the relationships between the identified inputs. – Specify valid values for each input. – Implement error handling for invalid inputs. 3. Handle Missing Inputs: – For each input, determine if it's mandatory or optional. – For mandatory inputs, implement user prompts to collect the information. – For optional inputs, define default values and explain their implications. 4. Data Structure Design: – Choose appropriate data structures to store and manage the inputs efficiently. – Ensure the chosen structures allow for easy updates and modifications. 5. Implement Decision Logic: – Develop the core logic to determine the work registration status based on the inputs. – Ensure all possible scenarios and edge cases are covered. 6. Documentation and Structure: – Provide clear, comprehensive comments throughout the code. – Organize the code into logical sections or functions for better readability. 7. Rule Storage: – Store decision-making rules in a format that's easily updatable (e.g., JSON, YAML). – Implement a mechanism to load and apply these rules dynamically. 8. Testing and Validation: – Include test cases covering various scenarios. – Validate the script's output against manual calculations. **Policy Documentation** {policy_text} Provide your Python script, following the steps outlined above

C. Design 3: Iterative Prompts

C. 設計三：迭代提示

To gain a better understanding of the basic tasks that make up the code generation process, we implemented an iterative prompting strategy, using the following steps:

> 為了更了解構成程式碼生成過程的基本任務，我們實作了一種迭代提示策略，步驟如下：

Expert knowledge construction: We summarized the policy excerpt to extract and distill key rules and requirements. This was a textual summary, not an explicit template as described in Experiment 3. The generated summary served as a condensed representation of the policy text, directing the LLM's focus toward only the essential information for subsequent steps.

> 專家知識建構：我們摘要政策摘錄，擷取並提煉出關鍵規則與要求。這是一份文字摘要，並非實驗三所述的明確範本。所生成的摘要是政策文字的濃縮表示，引導 LLM 在後續步驟中只專注於必要資訊。

Pseudocode generation: Based on the summarized policy, the LLM was instructed to generate pseudocode. A pseudocode is a simplified, human-readable description of the logical steps required to implement a program. This intermediate representation can be more useful to an LLM to generate code than the policy itself.

> 偽代碼生成：依據摘要後的政策，指示 LLM 生成偽代碼。偽代碼是對實作程式所需邏輯步驟的簡化、人類可讀的描述。這種中間表示形式，對 LLM 生成程式碼而言，可能比政策本身更有用。

Pseudocode translation: We asked the LLM to translate the pseudocode into executable Python code. This step converted the pseudo-code into a functional program that determined the worker registration status of an applicant given their input information.

> 偽代碼轉譯：我們要求 LLM 將偽代碼轉譯成可執行的 Python 程式碼。這個步驟把偽代碼轉成可運作的程式，依申請人的輸入資訊判定其工作登記狀態。

In this iterative approach, we created a structured pipeline for code generation by inputting each step's output as input for the next step. We have made our prompt templates available in the Appendix.

> 在這種迭代做法中，我們建立了結構化的程式碼生成管線，把每個步驟的輸出作為下一步的輸入。我們已將提示範本放在附錄中。

This structured methodology allowed us to systematically evaluate the impact of different prompting techniques and evaluate how well the LLM adapts to varying levels of guidance and task complexity. Each approach produced a Python script, which was then manually evaluated based on key qualitative benchmarks: variable identification, input handling, output correctness, decision making, logical consistency, rule coverage, and code execution. We discuss these criteria in the findings section.

> 這種結構化方法讓我們能系統性地評估不同提示技巧的影響，並評估 LLM 如何適應不同程度的引導與任務複雜度。每種做法都產出一份 Python 腳本，再依下列關鍵質性基準進行人工評估：變數識別、輸入處理、輸出正確性、決策、邏輯一致性、規則涵蓋率與程式碼執行。我們將在發現一節討論這些標準。

Developments and Challenges

發展與挑戰

Prompt Complexity and Task Alignment. Crafting prompts proved to be a central challenge throughout the experiment and particularly in the iterative approach, where each stage required tailored prompts with varying input structure and output requirements. In our experimentation, overly strict prompts constrained adaptability, while more loose prompts led to reasoning gaps that LLMs struggled to bridge. Striking the right balance required extensive trial and refinement.

> 提示的複雜度與任務對齊。撰寫提示是整個實驗的核心挑戰，尤其是在迭代做法中，每個階段都需要量身訂做的提示，其輸入結構與輸出要求各不相同。在我們的實驗中，過於嚴格的提示限制了適應性，而較寬鬆的提示則造成 LLM 難以彌補的推理缺口。要取得適當的平衡，需要大量的嘗試與調整。

Lack of Expert-Curated Test Cases for Benchmarking. Our evaluation was limited by the absence of standardized test cases designed by policy experts. A structured test suite would enable a more systematic validation,

reducing reliance on qualitative assessment and improving comparability across different systems and prompting methods.

> 缺乏專家整理的基準測試案例。我們的評估受限於缺少由政策專家設計的標準化測試案例。結構化的測試集可以支持更系統性的驗證，減少對質性評估的依賴，並提高不同系統與提示方法之間的可比較性。

Isolating Policy Interpretation from Code Generation. In non-iterative approaches, errors were difficult to attribute. Policy misinterpretations and code synthesis errors were often entangled in the final output. A more systematic verification framework is needed to distinguish policy comprehension failures from issues in logical modeling and code implementation.

> 將政策詮釋與程式碼生成分開。在非迭代的做法中，錯誤難以歸因。政策誤讀與程式碼合成錯誤，常常在最終輸出中糾纏在一起。需要更系統化的驗證框架，來區分政策理解的失敗，與邏輯建模和程式碼實作上的問題。

Findings

發現

Table 28 summarizes the performance of the three approaches using qualitative metrics that evaluate the effectiveness of the generated code. At a high level, Design 2 was the most successful. We now discuss each design individually before summarizing our findings.

> 表 28 使用評估所生成程式碼成效的質性指標，彙整了三種做法的表現。大體而言，設計二最成功。在彙整發現之前，我們先逐一討論每個設計。

We have made prompts and generated code available in the Appendix.

> 我們已將提示與生成的程式碼放在附錄中。

Table 28: Performance of Designs Across Criteria

表 28：各設計在各項標準上的表現（表格內容保留英文原文）

Criteria	Design 1 (Simple Prompt)	Design 2 (Detailed Prompt)	Design 3 (Iterative Prompts)
Variable Identification	Partial	Good	Good
Input Handling	Poor	Partially good	Poor
Output Correctness	Incorrect	Partially correct	Incorrect
Decision Making	Poor	Partially good	Poor (mechanical)
Logical Consistency	Poor	Improved	Partially consistent
Rule Coverage	Partially covered	Improved	Partially covered
Code Execution	Runs (unstable)	Runs (Improved)	Doesn't run

Design 1: Baseline approach was limited in code synthesis despite having foundational rules extracting capabilities. In the baseline approach, the LLM demonstrated a reasonable capacity to identify relevant rules and

variables required to construct the script, including correctly assigning data types (e.g., int, bool) and effectively categorizing rules, such as distinguishing exemption rules from mandatory registrant rules. This indicated a foundational ability to extract rules from the policy excerpt. However, significant issues existed in the final code output. Variable naming inconsistencies led to logical errors in the code, resulting in runtime errors and incorrect eligibility classifications. For example, as shown in Figure 18, the rule for determining "head-of-household" eligibility for 16- and 17-year-olds as part of the mandatory registrant decision logic referenced a 'school_status' variable that was never collected in the program input, causing the program to exit with an error.

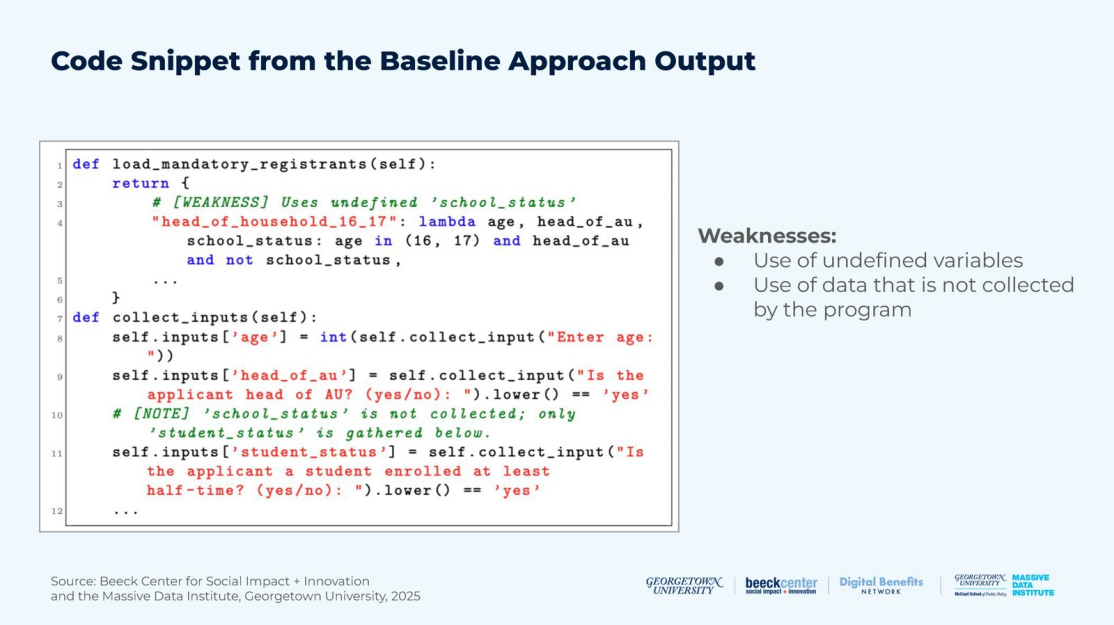
> 設計一：基準做法雖具備基礎的規則擷取能力，但在程式碼合成上受限。在基準做法中，LLM 展現出相當程度的能力，能找出建構腳本所需的相關規則與變數，包括正確指派資料型別（例如 int、bool），並有效分類規則，例如區分豁免規則與強制登記者規則。這顯示它具備從政策摘錄中擷取規則的基礎能力。然而，最終的程式碼輸出存在重大問題。變數命名不一致造成程式碼中的邏輯錯誤，導致執行階段錯誤與不正確的資格分類。例如，如圖 18 所示，在強制登記者判定邏輯中，用來判斷 16 與 17 歲者「戶長」資格的規則，引用了一個從未在程式輸入中蒐集的「school_status」變數，使程式因錯誤而結束。

Other issues causing runtime failures included the flawed input handling from a lack of proper validation and poor error-checking mechanisms. Additionally, failure to accurately model the policy logic led to inefficient code execution, such as requesting unnecessary user inputs rather than focusing on the subset needed for eligibility determination. In general, the LLM struggled to synthesize the multiple steps required—extracting relevant rules, interpreting their definitions, modeling interdependencies, and assembling the final code— to generate a coherent and functional output when provided with minimal guidance in the prompt.

> 造成執行失敗的其他問題，還包括因缺乏適當驗證與錯誤檢查機制而導致的輸入處理缺陷。此外，未能準確建模政策邏輯，造成程式碼執行效率不佳，例如要求使用者輸入不必要的資料，而不是聚焦於資格判定所需的那一部分。整體而言，在提示中只給予極少引導的情況下，LLM 很難綜合所需的多個步驟（擷取相關規則、詮釋其定義、建模相依關係，以及組裝最終程式碼），以產出連貫且可運作的輸出。

Figure 18: Code Snippet from the Baseline Approach Output

圖 18：基準做法輸出的程式碼片段

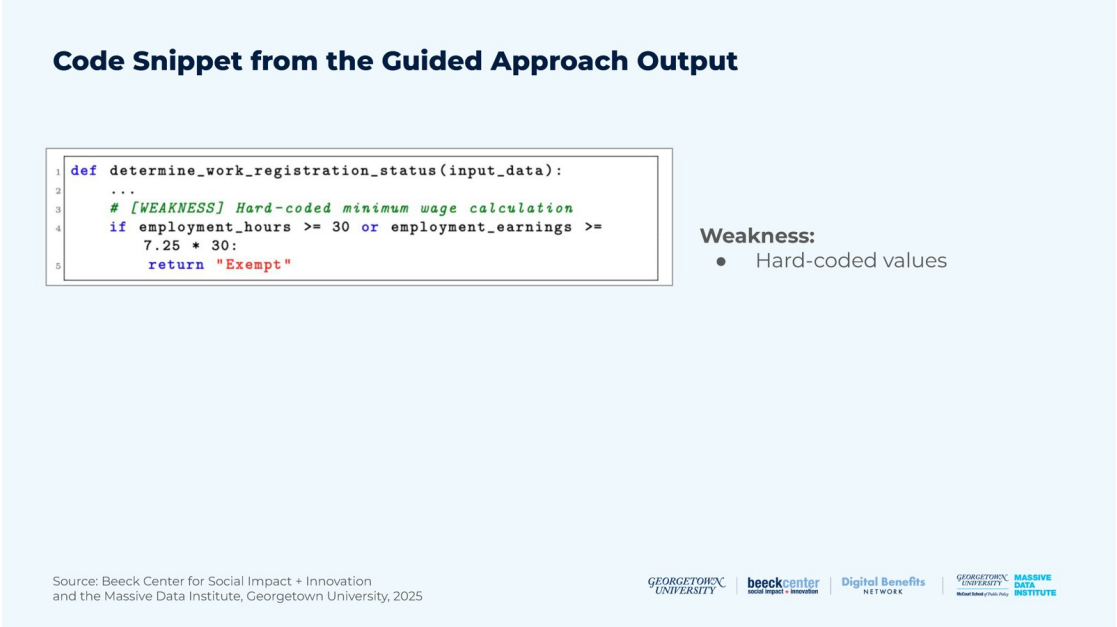


Design 2: The guided approach improved code quality but requires further refinement. With the guided approach, which incorporated detailed instructions, the LLM showed significant improvements over the baseline approach. The generated code demonstrated a more consistent logical structure, with variable naming and usage aligning throughout the program. The eligibility determination logic improved, and the resulting code was more accurate overall. However, certain limitations remained, including inadequate error handling mechanisms and a user interaction design that lacked clarity and intuitiveness. In addition, as shown in Figure 19, the eligibility logic still relies on hard-coded threshold values, limiting both code maintainability and adaptability to evolving policy and economic conditions. While the additional guidance enabled the LLM to better interpret and apply the policy rules, it still fell short of fully generating accurate, modular, adaptable code, highlighting areas where further refinement or human intervention may be necessary.

> 設計二：引導式做法改善了程式碼品質，但仍需進一步改進。在納入詳細指示的引導式做法下，LLM 相較於基準做法有顯著進步。所生成的程式碼具有較一致的邏輯結構，變數的命名與使用在整個程式中前後一致。資格判定邏輯有所改善，產出的程式碼整體上更準確。然而，仍存在某些限制，包括錯誤處理機制不足，以及使用者互動設計不夠清楚直觀。此外，如圖 19 所示，資格邏輯仍依賴寫死（hard-coded）的門檻值，限制了程式碼的可維護性，也限制了其適應不斷變化的政策與經濟狀況的能力。額外的引導雖然讓 LLM 更能詮釋並套用政策規則，但仍未能完全生成準確、模組化且可調適的程式碼，凸顯出可能需要進一步改進或人工介入的地方。

Figure 19: Code Snippet from the Guided Approach Output

圖 19：引導式做法輸出的程式碼片段



Design 3: Iterative approach introduced compounding error challenges despite task compartmentalization.

> 設計三：迭代做法儘管將任務分隔開來，仍帶來累積錯誤的挑戰。

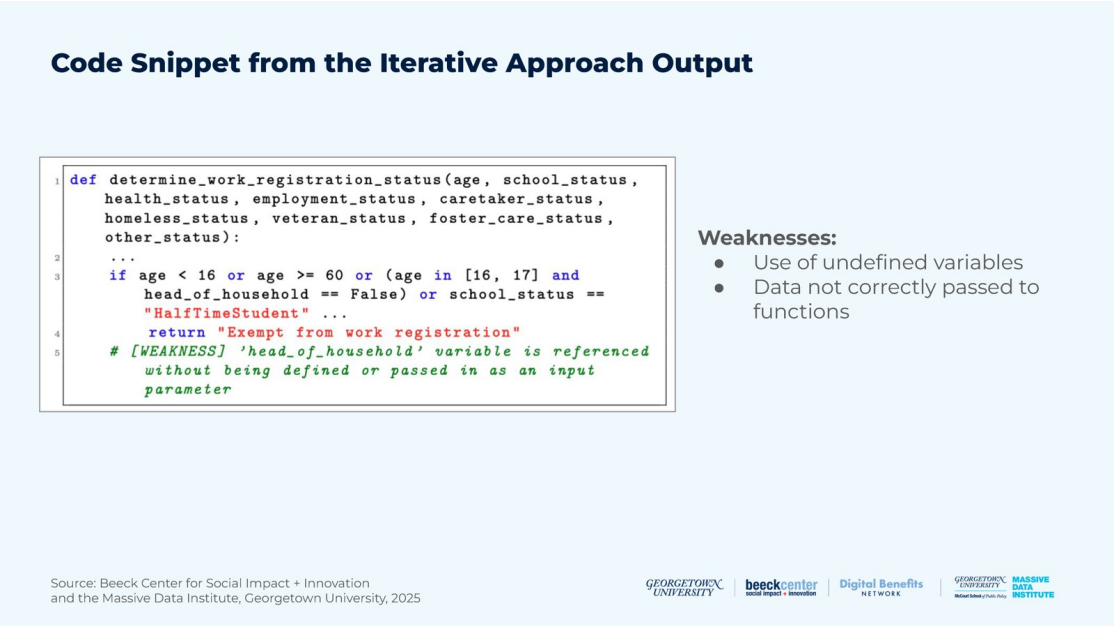
In the iterative approach, where the task was broken down into incremental steps—policy summarization, pseudo-code generation, and final code generation—the LLM showed promise in its ability to compartmentalize tasks. The policy summary effectively captured the document's structure and extracted key rules into a concise format, although there were occasional inconsistencies in articulating the eligibility logic. For example, the summary classified 51- to 59-year-olds as mandatory without mentioning they can be exempt if they are medically unfit, physically or mentally, or responsible for caring for a child under six. The pseudocode also formalized this summary into a logical framework but lacked precision, providing a simplified rather than comprehensive representation of the policy. For example, it treated every caretaker status as automatically exempt without verifying if the caretaker provided care for an incapacitated person or a child under six, overlooking essential policy requirements for eligibility checks. The final code, derived as a direct translation of the pseudocode, mirrored these deficiencies. The LLM's interpretation was mechanical, failing to fully grasp some of the policy logic embedded in the pseudocode. As a result, gaps in the pseudocode carried over to the final script, leading to execution errors. Key details were lost during the transitions between steps. For example, as shown in Figure 20, "head of household" was not explicitly defined as an input variable in the pseudocode, despite being an essential eligibility factor in the decision logic. This omission caused the generated code to reference an uninitialized variable, resulting in a runtime error. Contrary to our initial belief, this multi-step process introduced cumulative errors at each stage, proving less effective for generating functional code than Design 2.

> 在迭代做法中，任務被拆解成漸進的步驟（政策摘要、偽代碼生成與最終程式碼生成），LLM 在區隔任務的能力上展現出潛力。政策摘要有效掌握了文件結構，並將關鍵規則擷取成簡潔的格式，不過在闡述資格邏輯時偶爾出現不一致。例如，摘要把 51 至 59 歲者歸類為強制登記，卻沒有提到若他們在身體或精神上不適合工作，或需負責照顧六歲以下兒童，則可以豁免。偽代碼也將此摘要形式化為邏輯框架，

但缺乏精確度，提供的是簡化而非完整的政策呈現。例如，它把所有照顧者身分都視為自動豁免，卻沒有驗證照顧者所照顧的是否為失能者或六歲以下兒童，因而忽略了資格檢查的重要政策要求。最終的程式碼是偽代碼的直接翻譯，也延續了這些缺陷。LLM 的詮釋是機械式的，未能完全掌握偽代碼中蘊含的某些政策邏輯。因此，偽代碼中的缺口被帶入最終腳本，導致執行錯誤。關鍵細節在步驟之間的轉換中流失。例如，如圖 20 所示，「戶長」在偽代碼中並未被明確定義為輸入變數，儘管它是決策邏輯中的重要資格因素。這個遺漏使生成的程式碼引用了一個未初始化的變數，導致執行階段錯誤。與我們最初的想法相反，這種多步驟流程在每個階段都引入了累積的錯誤，證明它在生成可運作的程式碼上，不如設計二有效。

Figure 20: Code Snippet from the Iterative Approach Output

圖 20：迭代做法輸出的程式碼片段



In summary, our baseline experimental design generated unusable code, but did partially structure the logic effectively. The guided experimental design offered the strongest results for practical implementation when paired with human oversight. The prompts were refined and adapted to better align with policy rules, the underlying logic, and expected outputs. The iterative approach holds potential for complex and extended tasks; however, its utility depends on carefully mitigating errors that occur at each step of the pipeline through automated checks or expert review. Until more templates and standardized documents are developed, this approach is less viable than the guided approach.

總結而言，我們的基準實驗設計產出了無法使用的程式碼，但確實有效地部分建構了邏輯。引導式實驗設計在搭配人工監督時，為實際實施提供了最強的結果。提示經過精煉與調整，以更貼合政策規則、底層邏輯與預期輸出。迭代做法對複雜且冗長的任務具有潛力；然而，它的效用取決於是否能透過自動化

> 檢查或專家審查，審慎減少管線每個步驟中發生的錯誤。在開發出更多範本與標準化文件之前，這種做法的可行性不如引導式做法。

Considerations for Public Benefits Use Cases

對公共福利使用情境的考量

Summarized policy guidance for code generation reduces code errors. Our guided approach improved code quality significantly when compared to the baseline and also performed better than the iterative approach. Public benefits agencies should collaborate with each other to develop prompting frameworks that specify input format, variable naming conventions, and expected output structure to reduce ambiguities in LLM tasks, thereby improving the quality of the generated code.

> 為程式碼生成提供經過摘要的政策指引，可減少程式碼錯誤。相較於基準做法，我們的引導式做法大幅改善了程式碼品質，表現也優於迭代做法。公共福利機關應彼此合作，發展提示框架，明訂輸入格式、變數命名慣例與預期輸出結構，以減少 LLM 任務中的模糊之處，從而提升所生成程式碼的品質。

Modular design is particularly important in LLM workflows. Our findings show that LLMs struggle when they have to synthesize end-to-end logic for complex, multi-step tasks. Although their overall performance in the iterative approach was not markedly better than other methods, they showed a clear pattern of improved response quality when handling individual, compartmentalized tasks (e.g., summarizing policy). This suggests that public benefits agencies might potentially split complex workflows into smaller, reusable modules, which LLMs can tackle incrementally. At this stage, human developers are needed to both break down the workflow into reasonable chunks for LLMs and validate outputs from one step to the next.

> 模組化設計在 LLM 工作流程中特別重要。我們的發現顯示，當 LLM 必須綜合複雜、多步驟任務的端到端邏輯時，會顯得吃力。雖然它們在迭代做法中的整體表現並未明顯優於其他方法，但在處理個別、區隔開來的任務（例如摘要政策）時，回應品質呈現出明顯改善的規律。這顯示，公共福利機關或許可以把複雜的工作流程拆成較小、可重複使用的模組，讓 LLM 逐步處理。現階段，需要人類開發者既把工作流程拆成適合 LLM 處理的合理區塊，也驗證各步驟之間的輸出。

Accuracy and equity considerations must outweigh efficiency in high-stakes benefits systems. In code generation tasks—where outputs such as applicant eligibility status carry serious consequences and unresolved automation risks—we suggest treating LLMs as assistive tools to support human experts and developers, not as standalone solutions. Precision and fairness should be firm priorities in public benefits systems, making human oversight essential to ensure equitable outcomes. Efficiency gains from automation should not compromise these priorities.

> 在高風險的福利系統中，準確性與公平性的考量必須優先於效率。在程式碼生成任務中，申請人的資格狀態等輸出攸關重大後果，且自動化風險尚未解決，我們建議把 LLM 視為輔助人類專家與開發者的工

> 具，而非獨立的解決方案。精確與公平應成為公共福利系統的堅定優先事項，因此人類監督對確保公平的結果至關重要。自動化帶來的效率提升，不應損及這些優先事項。

Conclusion

結論

Key Takeaways

重點摘要

• Our experiments highlight that LLMs can support different parts of the Rules as Code pipeline, but a human in the loop and rich databases containing relevant, up-to-date policy excerpts are essential to facilitate the use of this technology.

> • 我們的實驗凸顯，LLM 能支持「規則即程式碼」流程的不同部分，但要促成這項技術的運用，人類參與其中，以及含有相關、最新政策摘錄的豐富資料庫，都不可或缺。

• Using LLMs to distill benefits eligibility policies leads to mixed results. The returned information varies in its accuracy, timeliness, state-specificity, completeness, and relevance. The results are mixed across models and interfaces, including the web browser and APIs. While one model may perform better using a specific prompt type (e.g., income or individual eligibility) or category of scoring (e.g., accuracy or relevance), it may perform poorly with other prompts and categories, leading to mixed overall performance.

> • 運用 LLM 提煉福利資格政策，結果好壞參半。回傳資訊在準確性、時效性、州別特定性、完整性與相關性上各不相同。結果在不同模型與介面（包括網頁瀏覽器與 API）之間也好壞不一。某個模型也許在特定提示類型（例如收入或個別資格）或評分類別（例如準確性或相關性）上表現較佳，卻可能在其他提示與類別上表現不佳，導致整體表現參差。

• Mixed LLM results have a direct impact on people seeking or receiving benefits, risking incorrect information when they ask generative AI models questions about programs like SNAP and Medicaid. When testing chatbots in Experiment 1, we found that Medicaid responses scored slightly higher overall than SNAP responses and that there were no major differences in the models we tested. Chatbots access information from the model's training data, and it is often unclear if that information is from authoritative sources. This means that we should expect some answers from chatbots to be inaccurate and therefore, encourage people to not use chatbots to obtain information about benefits programs if a source for the information is not returned.

> • LLM 結果好壞參半，對尋求或領取福利的民眾有直接影響：當他們向生成式 AI 模型詢問 SNAP 與 Medicaid 等計畫的問題時，有得到不正確資訊的風險。在實驗一測試聊天機器人時，我們發現 Medicaid 的回應整體得分略高於 SNAP，且所測試的模型之間沒有重大差異。聊天機器人從模型的

> 訓練資料取得資訊，而這些資訊是否來自權威來源往往並不清楚。這意味著，我們應預期聊天機器人的部分答案並不準確，因此，若沒有回傳資訊來源，應鼓勵民眾不要用聊天機器人取得福利計畫的資訊。

• It is possible to improve the performance of the benefits-related responses by pointing LLMs to authoritative sources like policy manuals. However, the current methods for communicating SNAP and Medicaid policy in fragmented PDFs and interactive websites make it hard for LLMs to use the information. State governments can make it easier for LLMs to use their policies by making them digitally accessible. At a minimum, this means a single PDF that allows text extraction; even better is a plain text or HTML document, or a webpage presenting the policy in full, including the effective date.

> ・ 把 LLM 指向政策手冊等權威來源，有可能改善福利相關回應的表現。然而，目前 SNAP 與 Medicaid 政策以分散的 PDF 與互動式網站傳達的方式，讓 LLM 難以使用這些資訊。州政府可以把政策做成數位化、易於取用的形式，讓 LLM 更容易使用其政策。最基本的做法是提供可擷取文字的單一 PDF；更好的是純文字或 HTML 文件，或完整呈現政策（包括生效日期）的網頁。

• When AI models provide incorrect information, they often do so in a confident tone, which can mislead those without subject expertise. This highlights the need for a standard way to quantify the uncertainty in AI responses, particularly for policy Rules as Code generation, where accuracy is essential.

> ・ 當 AI 模型提供不正確的資訊時，往往是以有把握的口氣呈現，這可能誤導缺乏專業知識的人。這凸顯了需要一種標準方式來量化 AI 回應中的不確定性，特別是對於準確性至關重要的政策「規則即程式碼」生成。

• Asking LLMs to write policy code directly leads to poor code. However, a guided approach with attached policy excerpts and very detailed prompts yields better executable code, though it remains poorly-designed.

> ・ 要求 LLM 直接撰寫政策程式碼，會得到品質不佳的程式碼。然而，附上政策摘錄並搭配非常詳細提示的引導式做法，可產生較好的可執行程式碼，但設計仍然不佳。

• Code generation can be improved by using an LLM with RAG to generate machine-readable policy rules. This is a viable alternative to manual rule curation. However, a manually-curated template is still necessary.

> ・ 運用 LLM 搭配 RAG 生成機器可讀的政策規則，可以改善程式碼生成。這是人工整理規則之外的可行替代方案。不過，人工整理的範本仍然是必要的。

Potential for Future Experimentation

未來實驗的潛力

We encourage ongoing, open experimentation to test the application of LLMs in public benefits eligibility and the development of Rules as Code.

> 我們鼓勵持續進行開放的實驗，以測試 LLM 在公共福利資格以及「規則即程式碼」發展上的應用。

• Our methodologies can be repeated for other states, programs, policies, or LLMs. Given that our research captures LLMs' performance at the time of our experiments, repeating these experiments as new versions are released can identify if and how performance improves with updates. Different prompting techniques, such as active prompting, may also improve LLM performance in Experiments 1 and 2. All experiments can also be applied to additional states, programs, and policies.

> • 我們的方法可以用於其他州、計畫、政策或 LLM 重複進行。鑑於我們的研究記錄的是實驗當時 LLM 的表現，隨著新版本推出而重複這些實驗，可以找出表現是否以及如何隨更新而改善。不同的提示技巧（例如主動提示，active prompting）也可能提升 LLM 在實驗一與實驗二中的表現。所有實驗也可以應用於更多的州、計畫與政策。

• Policy experts and software engineers can pair programs and evaluate. We conducted these experiments with general policy knowledge and computer science students. We've been following Nava Labs' project to work closely with benefits navigators and their design and engineering teams to develop new tools for benefits access and enrollment. Working with policy experts would be fruitful for capturing policy logic templates more effectively.

> • 政策專家與軟體工程師可以配對合作並評估。我們是以一般政策知識與資訊工程學生進行這些實驗。我們一直在關注 Nava Labs 的專案，他們與福利領航員（benefits navigators）及其設計與工程團隊密切合作，開發福利取得與申請的新工具。與政策專家合作，將有助於更有效地掌握政策邏輯範本。

• Explore creating a LLM for a specific program or policy. Creating specific LLMs fine-tuned for a specific program—such as SNAP—and continually updated as policies change with a RAG is an important direction, allowing states to have a stronger starting point in this space. Additionally, a program-specific model could have increased capabilities to understand differences between federal and state policy.

> • 探索為特定計畫或政策建立專用 LLM。針對特定計畫（例如 SNAP）微調專用 LLM，並搭配 RAG 隨政策變動持續更新，是一個重要方向，可讓各州在這個領域有更強的起點。此外，針對特定計畫的模型，可能更有能力理解聯邦與州政策之間的差異。

• Consider different programming languages for outputs. Experiment 4 could be explored using different programming languages to see if some LLMs are more or less successful at generating code.

> • 考慮用不同的程式語言作為輸出。實驗四可以改用不同的程式語言探索，看看某些 LLM 在生成程式碼上是否較成功或較不成功。

• Compare results against existing systems. Administrators of eligibility and enrollment systems could compare results against the code and information provided by those systems. Additionally, many existing systems also have test cases that could be used to evaluate LLM outputs.

> ・ 將結果與現有系統比較。資格審查與申請系統的管理者，可以將結果與這些系統所提供的程式碼與資訊相比較。此外，許多現有系統也有可用於評估 LLM 輸出的測試案例。

• Explore extracting code from legacy systems. Many eligibility and enrollment systems are written in legacy code languages. Administrators could explore using LLMs to convert the legacy code into an updated language using a guided template.

> ・ 探索從舊系統擷取程式碼。許多資格審查與申請系統是以舊的程式語言寫成。管理者可以探索運用 LLM，搭配引導式範本，把舊程式碼轉成較新的語言。

• Compare code writing efforts to policy analysis efforts. It is unclear how much effort using LLMs can save when developing Rules as Code infrastructures for different programs. An analysis of this may highlight a shift from programmer time to policy analyst time.

> ・ 比較撰寫程式碼的投入與政策分析的投入。目前尚不清楚在為不同計畫發展「規則即程式碼」基礎設施時，使用 LLM 能節省多少心力。對此的分析，或許能凸顯從程式設計師的時間轉移到政策分析師時間的變化。

Get in Touch

聯絡我們

We're eager to hear from you. This work, like any public-facing project, improves as more stakeholders offer their input. We welcome your thoughts, questions, or potential collaboration ideas. Email us at rulesascode@georgetown.edu.

> 我們很期待聽到你的意見。這項工作和任何面向公眾的專案一樣，隨著更多利害關係人提供意見而更臻完善。我們歡迎你的想法、問題或潛在的合作構想。請來信 rulesascode@georgetown.edu。

Citation

引用方式

Cite as: Ariel Kennan, Lisa Singh, Alessandra Garcia Guevara, Mohamed Ahmed, Jason Goodman, "AI-Powered Rules as Code: Experiments with Public Benefits Policy." Beeck Center for Social Impact + Innovation and Massive Data Institute, Georgetown University, March 24, 2025.

> 引用格式：Ariel Kennan, Lisa Singh, Alessandra Garcia Guevara, Mohamed Ahmed, Jason Goodman，"AI-Powered Rules as Code: Experiments with Public Benefits Policy"（AI 驅動的規則即程式碼：公共福利政策實驗），喬治城大學 Beeck 社會影響與創新中心（Beeck Center for Social Impact + Innovation）暨大數據研究院（Massive Data Institute），2025 年 3 月 24 日。

Icons from NounProject and Icons8.

Thank You

致謝

We'd like to acknowledge and thank the following colleagues for their contributions and support of this research:

> 我們謹向以下同仁對本研究的貢獻與支持致謝：（名單保留原文）

- Anjelika Deogirikar Grossman, Associate Director, MDI
- Ashlee Sellung, Digital Government Hub Coordinator
- Elizabeth Bynum Sorrell, Senior Research + Engagement Manager, DBN
- Jennifer Melot, Center for Security and Emerging Technology (CSET)
- Lynn Overmann, Executive Director, Beeck Center
- Olivia Zhao, Student Analyst, DBN
- Michael Ribar, Policy2Code Coach, Principal Associate for Social Policy and Economics Research at Westat
- Mahendran Velauthapillai, Professor and McBride Family Endowed Chair, Georgetown University Department of Computer Science
- Sabrina Toppa and Ashleigh Fryer, Beeck Center Communications Team
- Steve Kent, Software Developer, MDI
- Tina Amper, Community Advisor, Rules As Code Community of Practice, DBN

Appendix

附錄

Experiment materials are available in a public shared folder.

> 實驗資料可於公開共享資料夾取得。（附錄的評分準則、試算表與提示範本，本譯本僅列標題，請見原文。）

Experiment 1 Materials

實驗一資料

Experiment 2 Materials

實驗二資料

Experiment 3 Materials

實驗三資料

• Spreadsheet: Contains the generated rules from both the plain prompting and RAG approaches, compared against the verified SNAP Policy values in Georgia, Pennsylvania, and Texas.

> • 試算表：內含一般提示與 RAG 兩種做法所生成的規則，並與喬治亞州、賓夕法尼亞州、德州經驗證的 SNAP 政策數值相比對。

Experiment 4 Materials

實驗四資料

• Prompts: Includes the system and user prompts used for code generation. For the iterative approach, there are three sets of prompts for each stage, as outlined in the experiment details.

> • 提示：包含用於程式碼生成的系統提示與使用者提示。迭代做法的三個階段各有一組提示，如實驗細節所述。

• Generated Code: Contains the code results from each approach, with our manual annotations highlighting strengths [STRENGTH] and weaknesses [WEAKNESS].

> • 生成的程式碼：包含每種做法產出的程式碼結果，並附上我們的人工註記，以 [STRENGTH]（優點）與 [WEAKNESS]（缺點）標示。

Footnotes

註腳（參考文獻保持原樣，不翻譯）

• Ariel, Kennan, Lisa Singh, Bianca Dammholz, Keya Sengupta, and Jason Yi. “Exploring Rules Communication: Moving Beyond Static Documents to Standardized Code for U.S. Public Benefits Programs”, June 6, 2023. ↩

• Ariel Kennan and Tina Amper. “Cross-Sector Insights From the Rules as Code Community of Practice”, Digital Benefits Network, February 6, 2024. ↩

• James Mohun and A. Roberts, “Cracking the code: Rulemaking for humans and machines”, OECD Working Papers on Public Governance, No. 42, OECD Publishing, Paris, 2020. ↩

• Hamish Fraser and Tom Barraclough. Governing Digital Legal Systems: Insights on Artificial Intelligence and Rules as Code. MIT Computational Law Report, 2024. ↩

• Tom Barraclough, Hamish Fraser and Curtis Barnes. Legislation as Code, March 2021. ↩

• Denis Merigoux, Marie Alauzen, Lilya Slimani. Rules, Computation and Politics. Scrutinizing Unnoticed Programming Choices in French Housing Benefits. Journal of Cross-disciplinary Research in Computational Law, 2023, 2 (1), pp.23. ↩

• Justin D. Weisz, Michael Muller, Steven I. Ross, Fernando Martinez, Stephanie Houde, Mayank Agarwal, Kartik Talamadupula, and John T. Richards. 2022. Better Together? An Evaluation of AI-Supported Code Translation. In 27th International Conference on Intelligent User Interfaces (IUI '22), March 22–25, 2022, Helsinki, Finland. ACM, New York, NY, USA, 35 pages. ↩

• Basem Aly. How to talk to AIs: Advanced Prompt Engineering. Emerging Technologies Knowledge Base. December 2023. ↩

• Autumn Toney-Wails and Lisa Singh. Are You Sure About That: Eliciting Natural Language Confidence Indicators from Chatbots. In Proceedings of the 12th International Conference on Human-Agent Interaction (HAI '24). Association for Computing Machinery, New York, NY, USA, 278–286. ↩

• Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-Augmented Generation for Large Language Models: A Survey, March 2024. ↩